Code Lines for Image Handling

https://gemini.google.com/app/5f9d971367cb1826

User prompt: give me the lines pertaining to images

Response: The lines pertaining to image loading, rendering, and layout in `FINAL PROJECT DRAFT.py` are:

- **Importing Drafter's `Image` component (Lines 32–33)**

```
import drafter as _drafter              # optional layout / image / styling helpers
Image = getattr(_drafter, "Image", None)
```

- **Image directory and helper functions (Lines 199–226)**

```
# ---------------------------------------------------------------------------
# Card images and the table layout
# ---------------------------------------------------------------------------
# The images live in an "images" folder next to this file (create them with
# make_card_images.py).  images/cards/AS.png = ace of spades, TH.png = ten of hearts, ...
IMAGE_DIR = "images"
SUIT_LETTER = {"♠": "S", "♥": "H", "♦": "D", "♣": "C"}
...
def card_file(card: str) -> str:
    return IMAGE_DIR + "/cards/" + card[0] + SUIT_LETTER[card[1]] + ".png"


def card_image(card: str, width: int = 62):
    """A picture of the card (plain colored text if images are unavailable)."""
    if Image is None:
        return card_piece(card)
    return Image(card_file(card), width=width)


def back_image(width: int = 62):
    return Image(IMAGE_DIR + "/back.png", width=width) if Image is not None else "[??]"


def empty_image(width: int = 62):
    return Image(IMAGE_DIR + "/empty.png", width=width) if Image is not None else "[  ]"


def badge_image(kind: str, width: int = 30):
    """kind is 'dealer', 'small_blind' or 'big_blind'."""
    return Image(IMAGE_DIR + "/badges/" + kind + ".png", width=width) if Image is not None else None
```

- **Assembling card rows with face, back, and empty slot images (Lines 252–262)**

```
def card_row(cards: List[str], width: int = 62, hidden: bool = False, slots: int = 0) -> list:
    """Components for a row of cards: faces, or card backs when hidden."""
    row = []
    for c in cards:
        if row:
            row.append(" ")
        row.append(back_image(width) if hidden else card_image(c, width))
    for _ in range(slots - len(cards)):
        if row:
            row.append(" ")
        row.append(empty_image(width))
    return row
```

- **Displaying role badge images and hole card rows in a player's seat (Lines 280–320)**

```
def seat_box(state: State, i: int, reveal: bool = False):
    """One player's seat: name, role badges, chips, bet, status and cards (as images)."""
    ...
    items = []
    for role in seat_roles(state, i):
        badge = badge_image(role)
        if badge is not None:
            items.append(badge)
            items.append(" ")
    ...
    if p.hole_cards and not p.is_folded:
        mucked = reveal and i == 0 and won == 0 and state.auto_muck
```

```
        face_up = (i == 0 and not mucked) or (reveal and i != 0)
        items.extend(card_row(p.hole_cards, 62, hidden=not face_up))
```

- **Displaying community card images on the table and winning hand images at showdown (Lines 346 and 814)**

```
# Inside table_layout():
middle = card_row(state.community_cards, 70, slots=5)

# Inside showdown_page():
content.append(box(card_row(state.winning_cards, 62), bg=FELT, pad="8px", align="center"))
```[cite: 1]

User prompt: right now images are not actually being displayed, how can I fix that

Response: There are three common reasons Drafter won't display images with your current setup: **missing local image files**, **relative path mismatches**, or **an older Drafter version falling back to text**.

Follow these steps in order to diagnose and fix the issue:

1. Generate or Verify Your Image Folder

Your code looks for an `images` folder right next to `FINAL PROJECT DRAFT.py`, and specifically expects `back.png`, `empty.png`, a `cards/` subfolder (`AS.png`, `TH.png`, etc.), and a `badges/` subfolder (`dealer.png`, `small_blind.png`, `big_blind.png`).

- Notice your own comment on line 201: `# (create them with make_card_images.py)`. If you haven't run `make_card_images.py` yet, or if the `images/` folder isn't in the exact same directory where you launch Python, the browser will show broken image icons.
- **Verification:** Check your project folder to confirm `images/back.png`, `images/empty.png`, `images/cards/AS.png`, and `images/badges/dealer.png` exist at those exact paths (note that Linux/macOS and web servers are case-sensitive, so `AS.png` must be uppercase).

2. Check Which Working Directory You Run the Script From

Drafter serves static local files relative to the **current working directory** in your terminal, not necessarily the folder where the `.py` file lives.

- If your file is in `Desktop/poker/` and you run `python Desktop/poker/FINAL\ PROJECT\ DRAFT.py` from your home folder, Drafter will look for `~/images/...` instead of `~/Desktop/poker/images/...`.
- **Verification:** `cd` directly into the folder containing `FINAL PROJECT DRAFT.py` before running `python "FINAL PROJECT DRAFT.py"`.

3. Check if `Image` Is Imported Properly (Silent Fallback)

Lines 32–33 use `getattr(_drafter, "Image", None)`. If your version of Drafter doesn't expose `Image` at the top level—or if you are seeing bracketed text like `[A♠]` and `[??]` instead of broken image icons—`Image` is evaluating to `None` and triggering your text fallback.

- Replace lines 32–33 with a direct import so Python will tell you immediately if `Image` is missing or named differently in your version of Drafter:

```
from drafter import Image, Div, change_background_color, change_padding, change_text_align, change_border
```

- **Verification:** Run the script. If you get an `ImportError`, run `pip install --upgrade drafter` in your terminal to update to a version that supports `Image`.

4. Inspect the Browser Console for the Exact URL Failing

If you see broken image icons on the page, Drafter is rendering `<img>` tags, meaning the file paths are slightly off or blocked:

1. Open your browser's Developer Tools (`F12` or `Right-click -> Inspect`) and go to the **Network** or **Console** tab.
2. Refresh the poker table page and look for `404 Not Found` errors on `.png` files.
3. Compare the URL the browser requested (e.g., `http://localhost:8080/images/cards/AS.png`) against your actual folder structure.

User prompt: I have a zip folder containg all necessary contents how do I implement that

Response: To make your code load those images, extract the zip file into the exact folder structure your script expects and run Python from that directory.

1. **Locate your Python file's folder** Find the exact folder where `FINAL PROJECT DRAFT.py` is saved on your computer. *Verification:* You can see `FINAL PROJECT DRAFT.py` inside the folder window.
2. **Extract the zip archive into that same folder** Right-click your `.zip` file and choose **Extract All...** (Windows) or double-click it (Mac) to unzip its contents into the folder alongside `FINAL PROJECT DRAFT.py`. *Verification:* The extracted files are now standard folders rather than a `.zip` archive.

3. **Verify the exact folder structure** Your script uses `IMAGE_DIR = "images"` and looks for specific subfolders and uppercase card filenames. Ensure there isn't an extra wrapper folder created during extraction (such as `images/images/...` or `poker-assets/images/...`). The layout next to your `.py` file must match this tree:

```
your_project_folder/
├── FINAL PROJECT DRAFT.py
└── images/
    ├── back.png
    ├── empty.png
    ├── badges/
    │   ├── big_blind.png
    │   ├── dealer.png
    │   └── small_blind.png
    └── cards/
        ├── 2C.png
        ├── ...
        └── AS.png
```

   Verification: Opening `images/cards/` shows 52 PNG files with uppercase names (like `AS.png` and `TH.png`), and `images/badges/` contains the 3 badge PNGs.

4. **Run the script from inside that project folder** Open your terminal or IDE, navigate into `your_project_folder` (e.g., `cd path/to/your_project_folder`), and run `python "FINAL PROJECT DRAFT.py"`. If you launch it from a different working directory, Drafter will look for `images/` in the wrong place. *Verification:* Start a game in your browser—the card faces, card backs, and dealer/blind badges now appear on the green table instead of broken image icons or bracketed text.

User prompt: should I put the extracted file and the .py file in a new file together?

Response: Yes — creating a brand-new folder (directory) and putting both `FINAL PROJECT DRAFT.py` and the extracted `images` folder inside it together is the cleanest, safest way to keep everything organized and ensure Drafter finds your files.

Here is how to set up that folder cleanly:

1. **Create a new project folder** Make a new folder on your Desktop or Documents (for example, `TexasHoldemPoker`). *Verification:* You have a clean, empty folder named `TexasHoldemPoker`.

2. **Move `FINAL PROJECT DRAFT.py` into that new folder** Drag or copy `FINAL PROJECT DRAFT.py` into `TexasHoldemPoker`. *Verification:* Opening `TexasHoldemPoker` shows `FINAL PROJECT DRAFT.py` directly inside it.

3. **Move the extracted `images` folder into that same new folder** Drag the extracted `images` folder into `TexasHoldemPoker` so it sits right next to `FINAL PROJECT DRAFT.py` (do not put the `.py` file *inside* the `images` folder—they should be side by side inside `TexasHoldemPoker`). *Verification:* When you open `TexasHoldemPoker`, you see two items side by side: `FINAL PROJECT DRAFT.py` and the `images` folder (containing `back.png`, `empty.png`, `cards/`, and `badges/`).

4. **Open and run the `.py` file from that new folder** Open `TexasHoldemPoker` in your code editor (such as VS Code, Thonny, or IDLE) or `cd` into it in your terminal, then run `FINAL PROJECT DRAFT.py`. *Verification:* When you start a hand in the browser, the card images and dealer/blind badges load cleanly on the green table.

User prompt: - does this program follow these requirements: At least 7 routes. - At least 5 Pages, or 10 FragmentsLinks to an external site. - , or a combination thereof. - The implementation of the State dataclass must have at least 4 fields. - At least one field must be a list of class instances or a dictionary of class instances. - This means that there must be a second dataclass. - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants. - At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type. - At least 3 meaningful if statements. - At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way. - The site should have a legitimate purpose or functionality. - No global variables (global constantsLinks to an external site. - are fine). - You must use the Drafter library. - Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

Response: Your program meets almost every requirement cleanly, with **two minor caveats** regarding global variables (aliases) and documentation of advanced Python features that an automated grader or strict rubric might flag.

Requirement	Status	Verification in `FINAL PROJECT DRAFT_2.py`
At least 7 routes	**Pass (9)**	`@route` is used on `index`, `start_new_game`, `deal_hand`, `player_action`, `trigger_bot_turns`, `advance_street`, `reset_stack`, `show_rules`, and `return_to_start`.
At least 5 Pages	**Pass (5)**	`welcome_page`, `table_page`, `showdown_page`, `game_over_page`, and `rules_page` all return `Page(...)`.
State dataclass (≥ 4 fields + class list/dict)	**Pass**	`State` has 26 fields, including `players: List[Player]`, which uses the second `@dataclass class Player`.

Requirement	Status	Verification in `FINAL PROJECT DRAFT_2.py`
≥ 4 **State fields modified in routes**	**Pass**	`start_new_game`, `deal_hand`, `player_action`, and `advance_street` modify `state.players`, `state.community_cards`, `state.pot`, `state.current_turn_index`, `state.stage`, etc.
At least 3 input fields	**Pass (9)**	Uses `TextBox` (`player_name`, `raise_amount`), `SelectBox` (`starting_stack`, `difficulty`, `action_choice`), and `CheckBox` (`auto_muck`).
At least 3 meaningful `if` statements	**Pass**	Multiple conditional branches control game logic (e.g., raise validation in `player_action`, fold/showdown checks in `check_progress`, and stage routing in `index`).
≥ 1 **loop over class instance attributes**	**Pass**	`betting_round_complete` loops over `state.players` checking `p.is_folded`, `p.chips`, and `p.current_bet` (and `highest_bet`, `deal_hand`, `advance_street` do as well).
Legitimate purpose & uses Drafter	**Pass**	Full 5-player Texas Hold'em web application built with `from drafter import *` and `start_server(State())`.
No global variables (constants fine)	**Watch out**	Lines 29–37 assign module-level aliases (`Span`, `change_color`, `Image`, `Div`, etc.) in lowercase/mixed-case rather than `ALL_CAPS`.
Advanced techniques documented	**Watch out**	Uses several standard-library and language features outside typical intro CS courses without explanatory comments.

Two Things to Clean Up Before Submitting

1. **Module-Level Lowercase Variables (Lines 29–37)** Even though `_drafter`, `Span`, `Image`, `Div`, and `change_background_color` are never reassigned during gameplay, an automated AST linter checking for "no global variables" often flags any module-level assignment that isn't `ALL_CAPS`.
    - **Fix:** Import them directly inside a `try/except ImportError` block (or remove the `getattr` indirection if your Drafter version supports them natively), or rename fallback flags to `ALL_CAPS` constants.
2. **Documenting Advanced Techniques** Your inline comments explain *poker strategy* very well, but if your rubric strictly requires explaining *coding techniques* not covered in class, add brief comments explaining:
    - `itertools.combinations` (used in `best_score` and `best_hand` to generate all 21 five-card subsets from 7 cards)
    - `collections.Counter` (used to count rank frequencies for pairs, trips, and full houses)
    - Tuple comparison and `*` unpacking (e.g., `return (7, *ordered)` in `score_five` so Python compares hand tiers and kickers lexicographically)
    - `random.gauss` and Monte Carlo simulation (`estimate_equity`)
    - `field(default_factory=list)` in `@dataclass` (to avoid mutable default list bugs across `State` instances)
    - `divmod()` (used in `resolve_showdown` to split pots and compute remainder chips)

User prompt: are you able to edit the code so that there are not global variables, and all advanced techniques are documented, additionally write docstrings for all functions

Response: Below is the updated `FINAL PROJECT DRAFT_2.py` with:

1. **Zero non-constant global variables** — replaced the lowercase module-level `getattr` and `try/except` variable assignments with direct imports from `drafter`, leaving only `ALL_CAPS` constants at module scope.
2. **Complete docstrings on every function and dataclass** — every helper, evaluator, AI decision function, pot-resolution function, route, and page now has a clear docstring.
3. **Thorough inline documentation of advanced Python techniques** — added clear comments explaining `dataclass field(default_factory=list)`, `itertools.combinations`, `collections.Counter`, tuple comparison & `*` unpacking, generator/set comprehensions, `lambda` sort keys, `random.gauss` & Monte Carlo sampling, `divmod()`, and `**style` keyword unpacking.

```
"""
Texas Hold'em Poker -- a Drafter website
========================================
Play one human against four computer opponents (5 players at the table).

Run:   pip install drafter
       python "FINAL PROJECT DRAFT_2.py"

Structure
---------
```

```
* Two dataclasses: Player and State (State holds players, community_cards,
  pot and current_turn_index, plus supporting fields the game needs).
* Nine routes: index, start_new_game, deal_hand, player_action,
  trigger_bot_turns, advance_street, reset_stack, show_rules, return_to_start.
* Five pages: welcome/setup, main table, showdown/hand results,
  game over/rebuy, rules & hand rankings.
"""
from dataclasses import dataclass, field
from itertools import combinations
from collections import Counter
from typing import List, Optional, Tuple
import random

# Direct imports from Drafter avoid module-level lowercase alias variables
from drafter import (
    Page, Header, HorizontalRule, LineBreak, TextBox, SelectBox, CheckBox,
    Button, Table, Span, Image, Div, route, start_server,
    change_color, change_background_color, change_padding,
    change_text_align, change_border,
)


# ---------------------------------------------------------------------------
# Constants (ALL_CAPS module-level constants only; no global variables)
# ---------------------------------------------------------------------------
RANKS = "23456789TJQKA"
SUITS = "♠♥♦♣"

RANK_NAMES = {2: "Two", 3: "Three", 4: "Four", 5: "Five", 6: "Six", 7: "Seven",
              8: "Eight", 9: "Nine", 10: "Ten", 11: "Jack", 12: "Queen",
              13: "King", 14: "Ace"}
RANK_PLURALS = {2: "Twos", 3: "Threes", 4: "Fours", 5: "Fives", 6: "Sixes",
                7: "Sevens", 8: "Eights", 9: "Nines", 10: "Tens", 11: "Jacks",
                12: "Queens", 13: "Kings", 14: "Aces"}

BETTING_STAGES = ("preflop", "flop", "turn", "river")
STAGE_LABELS = {"setup": "Setup", "preflop": "Pre-flop", "flop": "Flop",
                "turn": "Turn", "river": "River", "showdown": "Showdown",
                "gameover": "Game Over"}

NUM_PLAYERS = 5          # 1 human + 4 computer opponents

SUIT_COLORS = {"♦": "#ff8c00",    # diamonds - orange
               "♥": "#d40000",    # hearts   - red
               "♣": "#1b8a1b",    # clubs    - green
               "♠": "#0a1f6e"}    # spades   - dark blue

IMAGE_DIR = "images"
SUIT_LETTER = {"♠": "S", "♥": "H", "♦": "D", "♣": "C"}
FELT = "#0b5d34"          # table cloth
SEAT_BG = "#14703f"       # a player's seat
ACTING_BG = "#ffe27a"     # whose turn it is
WINNER_BG = "#ffd24a"     # winner of the hand
FOLDED_BG = "#4a5a52"     # folded players

ROLE_LABEL = {"dealer": "Dealer", "small_blind": "Small Blind", "big_blind": "Big Blind"}

# AI opponent behavior tuning parameters by difficulty level
BOT_PROFILES = {
    "Easy":   {"sims": 15, "noise": 0.12, "pre_noise": 1.5, "raise_k": 0.55, "fold_margin": 0.65,
               "bluff": 0.02, "cbet": 0.10, "trap": 0.00, "semi": 0.00, "reads": False,
               "read_floor": 0.0, "position": False, "pre_tight": -1.0, "raise_extra": 2.5,
               "limp_gap": 3.0, "call_loose": 2.0, "threebet": 13.0, "bluff3": 0.00},
    "Medium": {"sims": 45, "noise": 0.04, "pre_noise": 0.6, "raise_k": 0.45, "fold_margin": 0.95,
               "bluff": 0.06, "cbet": 0.40, "trap": 0.10, "semi": 0.04, "reads": True,
               "read_floor": 5.0, "position": True, "pre_tight": 0.0, "raise_extra": 0.5,
               "limp_gap": 1.5, "call_loose": 0.0, "threebet": 11.5, "bluff3": 0.00},
    "Hard":   {"sims": 100, "noise": 0.00, "pre_noise": 0.0, "raise_k": 0.38, "fold_margin": 1.05,
               "bluff": 0.10, "cbet": 0.55, "trap": 0.25, "semi": 0.08, "reads": True,
               "read_floor": 6.5, "position": True, "pre_tight": 0.0, "raise_extra": 0.0,
               "limp_gap": 1.0, "call_loose": 0.0, "threebet": 11.0, "bluff3": 0.06},
}

HAND_RANKINGS = [
```

```
        ["1", "Royal Flush", "A, K, Q, J, 10, all of the same suit", "A♠ K♠ Q♠ J♠ 10♠"],
        ["2", "Straight Flush", "Five cards in sequence, all of the same suit", "9♥ 8♥ 7♥ 6♥ 5♥"],
        ["3", "Four of a Kind", "Four cards of the same rank", "Q♠ Q♥ Q♦ Q♣ 4♠"],
        ["4", "Full House", "Three of a kind plus a pair", "J♠ J♥ J♦ 8♣ 8♠"],
        ["5", "Flush", "Any five cards of the same suit", "K♦ 10♦ 8♦ 5♦ 2♦"],
        ["6", "Straight", "Five cards in sequence, mixed suits (Ace is high or low)", "9♣ 8♦ 7♠ 6♥ 5♣"],
        ["7", "Three of a Kind", "Three cards of the same rank", "7♠ 7♥ 7♦ K♣ 2♠"],
        ["8", "Two Pair", "Two different pairs", "A♠ A♦ 5♥ 5♣ 9♠"],
        ["9", "One Pair", "Two cards of the same rank", "10♠ 10♥ A♦ 7♣ 3♠"],
        ["10", "High Card", "No combination - the highest card plays", "A♠ J♦ 8♣ 5♥ 3♠"],
    ]


    # ---------------------------------------------------------------------------
    # Dataclasses
    # ---------------------------------------------------------------------------
    @dataclass
    class Player:
        """Represents one seat at the poker table (human or AI bot)."""
        name: str
        chips: int
        # ADVANCED TECHNIQUE (dataclasses.field): default_factory=list creates a brand-new
        # empty list per Player instance so instances do not share a single mutable list.
        hole_cards: List[str] = field(default_factory=list)
        current_bet: int = 0
        is_folded: bool = False


    @dataclass
    class State:
        """Holds the complete persistent application state passed between Drafter routes."""
        # --- Core table fields ---
        players: List[Player] = field(default_factory=list)
        community_cards: List[str] = field(default_factory=list)
        pot: int = 0
        current_turn_index: int = 0

        # --- Hand & betting progression fields ---
        stage: str = "setup"
        deck: List[str] = field(default_factory=list)
        dealer: int = 2
        hand_number: int = 0
        small_blind: int = 5
        big_blind: int = 10
        min_raise: int = 10
        raises_this_street: int = 0
        acted: List[bool] = field(default_factory=list)
        contributed: List[int] = field(default_factory=list)
        starting_stack: int = 1000
        difficulty: str = "Medium"
        auto_muck: bool = False
        message: str = ""
        action_log: List[str] = field(default_factory=list)

        # --- Opponent modeling data tracked for the AI ---
        raised_flags: List[bool] = field(default_factory=list)
        preflop_aggressor: int = -1
        opp_raises: List[int] = field(default_factory=list)
        opp_actions: List[int] = field(default_factory=list)

        # --- Showdown & hand result fields ---
        result_headline: str = ""
        reveal: bool = False
        hand_names: List[str] = field(default_factory=list)
        winnings: List[int] = field(default_factory=list)
        winning_cards: List[str] = field(default_factory=list)
        winning_hand_name: str = ""
        last_pot: int = 0

        # --- Session statistics ---
        hands_played: int = 0
        hands_won: int = 0
        biggest_pot: int = 0
        rebuys: int = 0
```

```
    peak_chips: int = 0


# ------------------------------------------------------------------------
# Cards, text formatting, and image/layout helpers
# ------------------------------------------------------------------------
def new_deck() -> List[str]:
    """Build and shuffle a standard 52-card deck represented as two-character strings."""
    # Nested list comprehension generates all 13x4 = 52 rank-suit combinations
    deck = [r + s for r in RANKS for s in SUITS]
    random.shuffle(deck)
    return deck


def show_card(card: str) -> str:
    """Convert internal ten representation ('T♠') into human-readable display ('10♠')."""
    return card.replace("T", "10")


def styled(text: str, color: str):
    """Wrap text in a colored Drafter Span component."""
    return change_color(Span(text), color)


def rich(*parts):
    """Combine plain strings and styled Span components into a single inline Span."""
    # ADVANCED TECHNIQUE (*args variadic parameters & *parts unpacking): accepts any
    # number of positional arguments as a tuple and unpacks them into Span(*parts).
    if all(isinstance(p, str) for p in parts):
        return "".join(parts)
    return Span(*parts)


def card_piece(card: str):
    """Render a single card string as a suit-colored bracketed text token."""
    return styled("[" + show_card(card) + "]", SUIT_COLORS[card[1]])


def colorize(text: str):
    """Color every card token (such as 'A♠' or '10♦') inside a space-separated string."""
    parts = []
    for i, token in enumerate(text.split(" ")):
        if i:
            parts.append(" ")
        if token and token[-1] in SUIT_COLORS:
            parts.append(styled(token, SUIT_COLORS[token[-1]]))
        else:
            parts.append(token)
    return rich(*parts)


def cards_text(cards: List[str]):
    """Render a list of cards as suit-colored text pieces separated by spaces."""
    parts = []
    for i, c in enumerate(cards):
        if i:
            parts.append(" ")
        parts.append(card_piece(c))
    return rich(*parts)


def board_text(cards: List[str]):
    """Render up to 5 community cards as colored text with '[  ]' placeholders for undealt slots."""
    parts = []
    for i in range(5):
        if i:
            parts.append(" ")
        parts.append(card_piece(cards[i]) if i < len(cards) else "[  ]")
    return rich(*parts)


def card_file(card: str) -> str:
    """Map a card code like 'A♠' to its PNG filename ('images/cards/AS.png')."""
    return IMAGE_DIR + "/cards/" + card[0] + SUIT_LETTER[card[1]] + ".png"
```

```
def card_image(card: str, width: int = 62):
    """Return a Drafter Image component for the face-up card."""
    return Image(card_file(card), width=width)


def back_image(width: int = 62):
    """Return a Drafter Image component for a face-down card back."""
    return Image(IMAGE_DIR + "/back.png", width=width)


def empty_image(width: int = 62):
    """Return a Drafter Image component for an empty community-card slot."""
    return Image(IMAGE_DIR + "/empty.png", width=width)


def badge_image(kind: str, width: int = 30):
    """Return a Drafter Image component for 'dealer', 'small_blind', or 'big_blind'."""
    return Image(IMAGE_DIR + "/badges/" + kind + ".png", width=width)


def decorate(component, bg=None, color=None, pad=None, align=None, border=None):
    """Apply optional background, text color, padding, alignment, and border styles to a component."""
    try:
        if bg:
            component = change_background_color(component, bg)
        if color:
            component = change_color(component, color)
        if pad:
            component = change_padding(component, pad)
        if align:
            component = change_text_align(component, align)
        if border:
            component = change_border(component, border)
    except Exception:
        pass
    return component


def box(items, **style):
    """Wrap a list of Drafter components inside a styled Div container."""
    # ADVANCED TECHNIQUE (**kwargs keyword unpacking): collects arbitrary style keyword
    # arguments (bg, color, pad, align, border) into a dict and forwards them to decorate().
    return decorate(Div(*items), **style)


def card_row(cards: List[str], width: int = 62, hidden: bool = False, slots: int = 0) -> list:
    """Build a horizontal list of card images (face-up, face-down, or empty placeholders)."""
    row = []
    for c in cards:
        if row:
            row.append(" ")
        row.append(back_image(width) if hidden else card_image(c, width))
    for _ in range(slots - len(cards)):
        if row:
            row.append(" ")
        row.append(empty_image(width))
    return row


def seat_roles(state: State, i: int) -> List[str]:
    """Return the active button/blind role keys ('dealer', 'small_blind', 'big_blind') for seat i."""
    n = len(state.players)
    roles = []
    if i == state.dealer:
        roles.append("dealer")
    if i == (state.dealer + 1) % n:
        roles.append("small_blind")
    if i == (state.dealer + 2) % n:
        roles.append("big_blind")
    return roles
```

```
    def seat_box(state: State, i: int, reveal: bool = False):
        """Build the styled visual seat box for player i, including badges, stack, status, and cards."""
        p = state.players[i]
        in_betting = state.stage in BETTING_STAGES
        won = state.winnings[i] if (state.stage == "showdown" and i < len(state.winnings)) else 0
        acting = in_betting and i == state.current_turn_index

        items = []
        for role in seat_roles(state, i):
            items.append(badge_image(role))
            items.append(" ")
        items.append(p.name + (" (you)" if i == 0 else ""))
        items.append(LineBreak())
        roles = [ROLE_LABEL[r] for r in seat_roles(state, i)]
        if roles:
            items.append(" / ".join(roles))
            items.append(LineBreak())
        items.append("Chips: " + str(p.chips) + (("   Bet: " + str(p.current_bet)) if in_betting else ""))
        items.append(LineBreak())

        if p.is_folded:
            status = "Folded"
        elif won > 0:
            status = "★ WINNER +" + str(won)
        elif acting:
            status = "▶ Your turn" if i == 0 else "▶ Acting"
        elif p.chips == 0 and in_betting:
            status = "All-in"
        else:
            status = "In hand" if in_betting else ""
        if status:
            items.append(status)
            items.append(LineBreak())

        if p.hole_cards and not p.is_folded:
            mucked = reveal and i == 0 and won == 0 and state.auto_muck
            face_up = (i == 0 and not mucked) or (reveal and i != 0)
            items.extend(card_row(p.hole_cards, 62, hidden=not face_up))

        if p.is_folded:
            bg, fg = FOLDED_BG, "#d0d8d4"
        elif won > 0:
            bg, fg = WINNER_BG, "#1a1a1a"
        elif acting:
            bg, fg = ACTING_BG, "#1a1a1a"
        else:
            bg, fg = SEAT_BG, "white"
        return box(items, bg=bg, color=fg, pad="8px", align="center", border="2px solid #e8d9a8")


    def table_layout(state: State, reveal: bool = False):
        """Arrange the 5 player seats, pot display, and 5-card community board into a 3x3 Table."""
        pot_box = box([
            "Pot: " + str(state.pot if state.stage in BETTING_STAGES else state.last_pot) + " chips",
            LineBreak(),
            STAGE_LABELS.get(state.stage, state.stage) + "  -  Blinds " + str(state.small_blind) + "/" + str(state.big_blind
        ], bg=FELT, color="white", pad="6px", align="center")

        middle = card_row(state.community_cards, 70, slots=5)
        board_box = box(middle, bg=FELT, color="white", pad="8px", align="center")
        blank = box([" "], bg=FELT)
        seats = {i: seat_box(state, i, reveal) for i in range(len(state.players))}
        rows = [
            [seats[2], pot_box, seats[3]],
            [seats[1], board_box, seats[4]],
            [blank, seats[0], blank],
        ]
        return decorate(Table(rows), bg=FELT, pad="10px", align="center")


    # ---------------------------------------------------------------------------
    # Hand evaluation and Monte Carlo equity estimation
    # ---------------------------------------------------------------------------
    def score_five(cards) -> Tuple[int, ...]:
```

```
        """Score a 5-card poker hand as a comparable tuple (category_rank, tiebreakers...)."""
        # Convert rank characters ('2'..'A') to integers 2..14 sorted high-to-low
        ranks = sorted((RANKS.index(c[0]) + 2 for c in cards), reverse=True)
        # Set comprehension checks whether all 5 cards share a single suit
        flush = len({c[1] for c in cards}) == 1
        # ADVANCED TECHNIQUE (collections.Counter + lambda key): Counter tallies rank
        # frequencies; sorting by (count, rank) groups pairs/trips/quads ahead of kickers.
        counts = Counter(ranks)
        groups = sorted(counts.items(), key=lambda kv: (kv[1], kv[0]), reverse=True)
        shape = [n for _, n in groups]
        ordered = [r for r, _ in groups]

        straight_high = 0
        if len(counts) == 5:
            if ranks[0] - ranks[4] == 4:
                straight_high = ranks[0]
            elif ranks == [14, 5, 4, 3, 2]:        # Ace-low straight ("wheel": A-2-3-4-5)
                straight_high = 5

        # ADVANCED TECHNIQUE (tuple comparison & * unpacking): Python compares tuples
        # lexicographically element-by-element, so higher hand tiers and higher kickers win.
        if straight_high and flush:
            return (9 if straight_high == 14 else 8, straight_high)
        if shape == [4, 1]:
            return (7, *ordered)
        if shape == [3, 2]:
            return (6, *ordered)
        if flush:
            return (5, *ranks)
        if straight_high:
            return (4, straight_high)
        if shape == [3, 1, 1]:
            return (3, *ordered)
        if shape == [2, 2, 1]:
            return (2, *ordered)
        if shape == [2, 1, 1, 1]:
            return (1, *ordered)
        return (0, *ranks)


    def best_score(cards) -> Tuple[int, ...]:
        """Find the highest 5-card score achievable from 5 to 7 cards."""
        best = None
        # ADVANCED TECHNIQUE (itertools.combinations): generates all 5-card subsets
        # from up to 7 available cards (7 choose 5 = 21 combinations) without manual loops.
        for five in combinations(cards, 5):
            s = score_five(five)
            if best is None or s > best:
                best = s
        return best


    def describe_hand(score: Tuple[int, ...]) -> str:
        """Translate a hand score tuple into a readable name such as 'Full House, Kings full of Tens'."""
        cat = score[0]
        if cat == 9:
            return "Royal Flush"
        if cat == 8:
            return "Straight Flush, " + RANK_NAMES[score[1]] + " high"
        if cat == 7:
            return "Four of a Kind, " + RANK_PLURALS[score[1]]
        if cat == 6:
            return "Full House, " + RANK_PLURALS[score[1]] + " full of " + RANK_PLURALS[score[2]]
        if cat == 5:
            return "Flush, " + RANK_NAMES[score[1]] + " high"
        if cat == 4:
            return "Straight, " + RANK_NAMES[score[1]] + " high"
        if cat == 3:
            return "Three of a Kind, " + RANK_PLURALS[score[1]]
        if cat == 2:
            return "Two Pair, " + RANK_PLURALS[score[1]] + " and " + RANK_PLURALS[score[2]]
        if cat == 1:
            return "Pair of " + RANK_PLURALS[score[1]]
        return "High Card, " + RANK_NAMES[score[1]]
```

```
    def best_hand(cards) -> Tuple[Tuple[int, ...], List[str], str]:
        """Return (best_score_tuple, ordered_5_cards, hand_description) from 5 to 7 cards."""
        best = None
        best_five = None
        for five in combinations(cards, 5):
            s = score_five(five)
            if best is None or s > best:
                best, best_five = s, list(five)
        counts = Counter(c[0] for c in best_five)
        best_five.sort(key=lambda c: (counts[c[0]], RANKS.index(c[0])), reverse=True)
        if best[0] in (4, 8) and best[1] == 5:     # place Ace at the end for A-2-3-4-5 straights
            best_five.append(best_five.pop(0))
        return best, best_five, describe_hand(best)


    def chen_score(cards) -> float:
        """Compute the Chen Formula pre-flop starting hand strength score (roughly -1 to 20)."""
        r1, r2 = sorted((RANKS.index(c[0]) + 2 for c in cards), reverse=True)
        top = {14: 10.0, 13: 8.0, 12: 7.0, 11: 6.0}.get(r1, r1 / 2.0)
        if r1 == r2:
            return max(5.0, top * 2)
        score = top
        if cards[0][1] == cards[1][1]:
            score += 2                                  # suited bonus
        gap = r1 - r2 - 1
        score -= {0: 0, 1: 1, 2: 2, 3: 4}.get(gap, 5)
        if gap <= 1 and r1 < 12:
            score += 1                                  # connectedness bonus below Queen
        return score


    def estimate_equity(hole: List[str], board: List[str], floors: List[float], sims: int) -> float:
        """Run a Monte Carlo simulation of remaining runouts to estimate win probability (0.0 to 1.0)."""
        # ADVANCED TECHNIQUE (Monte Carlo simulation & set difference): builds the unseen deck
        # via set membership and samples `sims` random board completions and opponent holdings.
        seen = set(hole) | set(board)
        deck = [r + s for r in RANKS for s in SUITS if (r + s) not in seen]
        need = 5 - len(board)
        wins = 0.0
        for _ in range(sims):
            d = deck[:]
            random.shuffle(d)
            full_board = board + d[:need]
            rest = d[need:]
            mine = best_score(hole + full_board)
            best_opp = None
            for floor in floors:
                picked = None
                if floor > 0:
                    for _try in range(12):
                        i, j = random.sample(range(len(rest)), 2)
                        if chen_score((rest[i], rest[j])) >= floor:
                            picked = (i, j)
                            break
                if picked is None:
                    picked = (len(rest) - 1, len(rest) - 2)
                hand = [rest[picked[0]], rest[picked[1]]]
                for k in sorted(picked, reverse=True):
                    rest.pop(k)
                sc = best_score(hand + full_board)
                if best_opp is None or sc > best_opp:
                    best_opp = sc
            if mine > best_opp:
                wins += 1
            elif mine == best_opp:
                wins += 0.5
        return wins / sims


    # ---------------------------------------------------------------------------
    # Betting-round helpers
    # ---------------------------------------------------------------------------
```

```
    def can_act(player: Player) -> bool:
        """Return True if the player has not folded and still has chips available to bet."""
        return (not player.is_folded) and player.chips > 0


    def highest_bet(state: State) -> int:
        """Return the highest current_bet among all non-folded players in the active round."""
        top = 0
        for p in state.players:
            if not p.is_folded and p.current_bet > top:
                top = p.current_bet
        return top


    def next_actor(state: State) -> int:
        """Return the seat index of the next active player who can act, wrapping around modulo N."""
        n = len(state.players)
        for step in range(1, n + 1):
            candidate = (state.current_turn_index + step) % n
            if can_act(state.players[candidate]):
                return candidate
        return state.current_turn_index


    def betting_round_complete(state: State) -> bool:
        """Return True when every active player has matched the highest bet (or is all-in) and acted."""
        highest = highest_bet(state)
        able = 0
        waiting = 0
        for i, p in enumerate(state.players):
            if p.is_folded or p.chips == 0:
                continue
            if p.current_bet != highest:
                return False
            able += 1
            if not state.acted[i]:
                waiting += 1
        if able >= 2 and waiting > 0:
            return False
        return True


    def add_log(state: State, text: str) -> None:
        """Append a human-readable action entry to the current hand's action log."""
        state.action_log.append(text)


    def put_chips(state: State, idx: int, amount: int) -> None:
        """Move up to `amount` chips from player idx's stack into their current bet and the pot."""
        p = state.players[idx]
        amount = min(amount, p.chips)
        p.chips -= amount
        p.current_bet += amount
        state.pot += amount
        state.contributed[idx] += amount


    def post_blind(state: State, idx: int, amount: int, label: str) -> None:
        """Deduct a forced blind bet from player idx and record it in the action log."""
        put_chips(state, idx, amount)
        p = state.players[idx]
        add_log(state, p.name + " posts the " + label + " (" + str(p.current_bet) + ")")


    def apply_action(state: State, idx: int, action: str, raise_by: int = 0) -> None:
        """Execute 'fold', 'call' (check/call), or 'raise' for player idx and update turn tracking."""
        p = state.players[idx]
        to_call = max(0, highest_bet(state) - p.current_bet)
        state.opp_actions[idx] += 1
        if action == "fold":
            p.is_folded = True
            add_log(state, p.name + " folds")
        elif action == "raise":
            put_chips(state, idx, to_call + raise_by)
```

```
            state.min_raise = max(state.min_raise, raise_by)
            state.raises_this_street += 1
            state.opp_raises[idx] += 1
            state.raised_flags[idx] = True
            if state.stage == "preflop":
                state.preflop_aggressor = idx
            for j in range(len(state.players)):
                if j != idx:
                    state.acted[j] = False
            note = " (all-in)" if p.chips == 0 else ""
            add_log(state, p.name + " raises to " + str(p.current_bet) + note)
        else:
            paid = min(to_call, p.chips)
            put_chips(state, idx, paid)
            if to_call == 0:
                add_log(state, p.name + " checks")
            else:
                note = " (all-in)" if p.chips == 0 else ""
                add_log(state, p.name + " calls " + str(paid) + note)
        state.acted[idx] = True


    # ------------------------------------------------------------------------
    # Computer opponent decision making
    # ------------------------------------------------------------------------
    def raise_rate(state: State, j: int) -> float:
        """Return player j's historical raise frequency (defaults to 0.25 until 8 actions observed)."""
        actions = state.opp_actions[j]
        return state.opp_raises[j] / actions if actions >= 8 else 0.25


    def bot_context(state: State, idx: int, profile: dict) -> dict:
        """Gather table geometry, bet sizes, opponent reads, and raise legality for bot idx."""
        bot = state.players[idx]
        n = len(state.players)
        highest = highest_bet(state)
        to_call = max(0, highest - bot.current_bet)
        room = bot.chips - to_call
        opps = [j for j, p in enumerate(state.players) if j != idx and not p.is_folded]

        lateness = ((idx - state.dealer - 1) % n) / (n - 1) if profile["position"] else 0.5

        agg, agg_scale = 0.0, 1.0
        if profile["reads"] and to_call > 0:
            aggressor = max(opps, key=lambda j: state.players[j].current_bet)
            rate = raise_rate(state, aggressor)
            if rate < 0.15:
                agg, agg_scale = 1.5, 1.15
            elif rate > 0.40:
                agg, agg_scale = -1.0, 0.92

        return {"highest": highest, "to_call": to_call, "room": room, "opps": opps,
                "lateness": lateness, "agg": agg, "agg_scale": agg_scale,
                "can_raise": room >= state.min_raise and state.raises_this_street < 3}


    def make_raise(state: State, ctx: dict, raise_by: int) -> Tuple[str, int]:
        """Clamp a desired raise increment to legal min/max bounds, or fall back to calling."""
        if not ctx["can_raise"]:
            return ("call", 0)
        raise_by = max(int(raise_by), state.min_raise)
        return ("raise", min(raise_by, ctx["room"]))


    def pot_raise(state: State, ctx: dict, fraction: float) -> Tuple[str, int]:
        """Size a bet or raise as a fraction of the pot rounded down to the small blind."""
        size = int((state.pot + ctx["to_call"]) * fraction)
        size = (size // state.small_blind) * state.small_blind
        return make_raise(state, ctx, size)


    def raise_to(state: State, ctx: dict, target_total: float) -> Tuple[str, int]:
        """Convert a desired total bet level into a legal raise increment above highest_bet."""
        return make_raise(state, ctx, int(target_total) - ctx["highest"])
```

```
def shove(ctx: dict) -> Tuple[str, int]:
    """Move all-in by raising all remaining chips if legal, otherwise calling."""
    return ("raise", ctx["room"]) if ctx["can_raise"] else ("call", 0)


def preflop_action(state: State, idx: int, profile: dict, ctx: dict) -> Tuple[str, int]:
    """Choose a bot's pre-flop action using Chen score, table position, and stack depth."""
    bot = state.players[idx]
    bb = state.big_blind
    to_call, highest = ctx["to_call"], ctx["highest"]
    n_active = len(ctx["opps"]) + 1
    late = ctx["lateness"]

    chen = chen_score(bot.hole_cards)
    # ADVANCED TECHNIQUE (random.gauss): adds Gaussian (normal distribution) noise
    # so Easy/Medium bots occasionally misjudge marginal starting hands.
    if profile["pre_noise"] > 0:
        chen += random.gauss(0, profile["pre_noise"])
    open_thr = 7.5 - 3.0 * late + 0.3 * (n_active - 3) + profile["pre_tight"]
    stack_bb = (bot.chips + bot.current_bet) / bb
    raised_pot = highest > bb
    check_or_fold = ("call", 0) if to_call == 0 else ("fold", 0)

    if stack_bb <= 10:
        push_thr = 9.5 if raised_pot else 6.0
        return shove(ctx) if chen >= push_thr else check_or_fold

    if not raised_pot:
        limpers = max(0, sum(1 for j in ctx["opps"] if state.players[j].current_bet >= bb) - 1)
        if chen >= open_thr + profile["raise_extra"]:
            return raise_to(state, ctx, bb * (3 + limpers))
        if to_call == 0 or (chen >= open_thr - profile["limp_gap"] and to_call <= bb):
            return ("call", 0)
        return ("fold", 0)

    is_bb = idx == (state.dealer + 2) % len(state.players)
    price = to_call / (state.pot + to_call)
    premium = 13.5 if state.raises_this_street >= 2 else profile["threebet"] + ctx["agg"]
    if chen >= premium:
        return raise_to(state, ctx, highest * (3 if late >= 0.5 else 3.5))
    if (state.raises_this_street == 1 and late >= 0.75 and 6 <= chen < premium
            and random.random() < profile["bluff3"]):
        return raise_to(state, ctx, highest * 3)

    call_thr = open_thr + 0.8 + 0.6 * max(0.0, to_call / bb - 2) + ctx["agg"] - profile["call_loose"]
    if price < 0.2:
        call_thr -= 1.5
    if is_bb and to_call <= 3 * bb:
        call_thr -= 1.5
    if to_call >= 0.4 * bot.chips:
        call_thr = max(call_thr, 11.0)
    call_thr = min(call_thr, 14.0)
    return ("call", 0) if chen >= call_thr else ("fold", 0)


def postflop_action(state: State, idx: int, profile: dict, ctx: dict) -> Tuple[str, int]:
    """Choose a bot's post-flop action using Monte Carlo equity, pot odds, and bluff frequencies."""
    bot = state.players[idx]
    to_call, room, late = ctx["to_call"], ctx["room"], ctx["lateness"]
    opps = ctx["opps"]
    n_opp = len(opps)

    floors = []
    for j in opps:
        floor = 0.0
        if profile["read_floor"] > 0 and state.raised_flags[j]:
            rate = raise_rate(state, j)
            floor = profile["read_floor"] + (1.5 if rate < 0.15 else (-1.5 if rate > 0.40 else 0.0))
        floors.append(floor)
    equity = estimate_equity(bot.hole_cards, state.community_cards, floors, profile["sims"])
    if profile["noise"] > 0:
        equity += random.gauss(0, profile["noise"])
```

```
        fair = 1.0 / (n_opp + 1)
        raise_at = fair + profile["raise_k"] * (1.0 - fair)
        pot_odds = to_call / (state.pot + to_call) if to_call > 0 else 0.0
        need = pot_odds * profile["fold_margin"] * ctx["agg_scale"]
        if profile["reads"] and to_call > 0 and state.stage in ("flop", "turn") and room > 2 * state.pot:
            need -= 0.03
        if to_call > 0 and to_call >= 0.6 * bot.chips:
            need = min(need, pot_odds)
        check_or_fold = ("call", 0) if to_call == 0 else ("fold", 0)

        if (bot.chips + bot.current_bet) / state.big_blind <= 8:
            if equity >= raise_at - 0.08:
                return shove(ctx)
            return ("call", 0) if (to_call == 0 or equity >= need) else check_or_fold

        if to_call == 0:
            if equity >= raise_at:
                if equity > 0.88 and state.stage != "river" and random.random() < profile["trap"]:
                    return ("call", 0)
                fraction = 0.5 if equity < 0.65 else (0.7 if equity < 0.8 else 1.0)
                return pot_raise(state, ctx, fraction)
            if (state.stage == "flop" and state.preflop_aggressor == idx and n_opp <= 2
                    and random.random() < profile["cbet"]):
                return pot_raise(state, ctx, 0.5)
            if n_opp <= 2 and 0.2 <= equity < raise_at and random.random() < profile["bluff"] * (0.5 + late):
                return pot_raise(state, ctx, 0.55)
            return ("call", 0)

        if equity < need:
            if (state.stage != "river" and n_opp == 1
                    and random.random() < profile["bluff"] * 0.4):
                return pot_raise(state, ctx, 0.8)
            return ("fold", 0)
        strong = equity >= raise_at + 0.05
        if state.raises_this_street >= 2:
            strong = equity >= 0.85
        if strong:
            return pot_raise(state, ctx, 0.75 if equity < 0.8 else 1.0)
        if (state.stage in ("flop", "turn") and n_opp == 1 and 0.3 <= equity < raise_at
                and random.random() < profile["semi"]):
            return pot_raise(state, ctx, 0.75)
        return ("call", 0)


    def choose_bot_action(state: State, idx: int) -> Tuple[str, int]:
        """Dispatch bot decision making to preflop_action or postflop_action based on game stage."""
        profile = BOT_PROFILES.get(state.difficulty, BOT_PROFILES["Medium"])
        ctx = bot_context(state, idx, profile)
        if state.stage == "preflop":
            return preflop_action(state, idx, profile, ctx)
        return postflop_action(state, idx, profile, ctx)


    # ------------------------------------------------------------------------
    # Pot awarding and showdown resolution
    # ------------------------------------------------------------------------
    def build_side_pots(state: State) -> List[Tuple[int, List[int]]]:
        """Partition total chips contributed into main and side pots for all-in scenarios."""
        pots = []
        previous = 0
        # Set comprehension extracts unique positive contribution thresholds in ascending order
        for level in sorted({c for c in state.contributed if c > 0}):
            contributors = [i for i, c in enumerate(state.contributed) if c >= level]
            amount = (level - previous) * len(contributors)
            eligible = [i for i in contributors if not state.players[i].is_folded]
            if not eligible:
                eligible = [i for i, p in enumerate(state.players) if not p.is_folded]
            pots.append((amount, eligible))
            previous = level
        return pots


    def finish_hand(state: State, winnings: List[int], reveal: bool, headline: str,
```

```
                hand_names: List[str], winning_cards: List[str], winning_name: str) -> Page:
    """Credit winners, zero the pot, update session statistics, and render showdown_page."""
    total = state.pot
    for i, amount in enumerate(winnings):
        state.players[i].chips += amount
    state.pot = 0
    state.winnings = winnings
    state.reveal = reveal
    state.result_headline = headline
    state.hand_names = hand_names
    state.winning_cards = winning_cards
    state.winning_hand_name = winning_name
    state.last_pot = total
    state.stage = "showdown"
    state.hands_played += 1
    if winnings[0] > 0:
        state.hands_won += 1
        state.biggest_pot = max(state.biggest_pot, total)
    state.peak_chips = max(state.peak_chips, state.players[0].chips)
    return showdown_page(state)


def award_uncontested(state: State, winner_idx: int) -> Page:
    """Award the entire pot to the sole remaining player when all opponents have folded."""
    n = len(state.players)
    winnings = [0] * n
    winnings[winner_idx] = state.pot
    winner = state.players[winner_idx]
    headline = "★ " + winner.name + " wins " + str(state.pot) + " chips - everyone else folded ★"
    return finish_hand(state, winnings, False, headline, [""] * n, [], "")


def resolve_showdown(state: State) -> Page:
    """Evaluate all non-folded hands at showdown, distribute main/side pots, and show results."""
    n = len(state.players)
    scored = {}
    for i, p in enumerate(state.players):
        if not p.is_folded:
            scored[i] = best_hand(p.hole_cards + state.community_cards)

    winnings = [0] * n
    for amount, eligible in build_side_pots(state):
        in_running = [i for i in eligible if i in scored]
        top = max(scored[i][0] for i in in_running)
        winners = [i for i in in_running if scored[i][0] == top]
        # Built-in divmod() computes integer split share and any indivisible odd-chip remainder
        share, extra = divmod(amount, len(winners))
        for w in winners:
            winnings[w] += share
        winnings[winners[0]] += extra

    hand_names = [scored[i][2] if i in scored else "" for i in range(n)]
    best_idx = max(scored, key=lambda i: scored[i][0])
    best_score_value = scored[best_idx][0]
    top_players = [i for i in scored if scored[i][0] == best_score_value]
    paid = [i for i in range(n) if winnings[i] > 0]

    best_name = scored[best_idx][2]
    if len(paid) == 1:
        headline = "★ " + state.players[paid[0]].name + " wins " + str(state.pot) + " chips with " + best_name + " ★"
    elif set(paid) == set(top_players):
        names = " and ".join(state.players[i].name for i in paid)
        headline = "★ Split pot: " + names + " tie with " + best_name + " ★"
    else:
        parts = [state.players[i].name + " wins " + str(winnings[i]) for i in paid]
        headline = "★ " + ", ".join(parts) + " (best hand: " + best_name + ") ★"
    return finish_hand(state, winnings, True, headline, hand_names, scored[best_idx][1], scored[best_idx][2])


def check_progress(state: State) -> Optional[Page]:
    """Check if the hand ended by folds or if the betting round is complete; return next Page or None."""
    active = [i for i, p in enumerate(state.players) if not p.is_folded]
    if len(active) == 1:
        return award_uncontested(state, active[0])
```

```
    if betting_round_complete(state):
        if len(state.community_cards) < 5:
            return advance_street(state)
        return resolve_showdown(state)
    return None


# ---------------------------------------------------------------------------
# Routes
# ---------------------------------------------------------------------------
@route
def index(state: State) -> Page:
    """Render the page corresponding to the current game stage (setup, table, showdown, or gameover)."""
    if state.stage == "setup":
        return welcome_page(state)
    if state.stage == "showdown":
        return showdown_page(state)
    if state.stage == "gameover":
        return game_over_page(state)
    return table_page(state)


@route
def start_new_game(state: State, player_name: str, starting_stack: str, difficulty: str) -> Page:
    """Initialize player seats, blind levels, and session statistics, then deal the first hand."""
    try:
        stack = int(starting_stack)
    except ValueError:
        stack = 1000
    name = player_name.strip() or "User"

    state.starting_stack = stack
    state.difficulty = difficulty if difficulty in BOT_PROFILES else "Medium"
    state.big_blind = max(2, (stack // 50) // 2 * 2)
    state.small_blind = state.big_blind // 2
    state.min_raise = state.big_blind

    state.players = [Player(name, stack)] + [Player("Bot " + str(k), stack) for k in range(1, NUM_PLAYERS)]
    n = len(state.players)
    state.community_cards = []
    state.pot = 0
    state.current_turn_index = 0
    state.deck = new_deck()
    state.dealer = n - 1
    state.hand_number = 0
    state.acted = [False] * n
    state.contributed = [0] * n
    state.raised_flags = [False] * n
    state.preflop_aggressor = -1
    state.opp_raises = [0] * n
    state.opp_actions = [0] * n
    state.action_log = []
    state.message = ""
    state.hands_played = 0
    state.hands_won = 0
    state.biggest_pot = 0
    state.rebuys = 0
    state.peak_chips = stack
    return deal_hand(state)


@route
def deal_hand(state: State) -> Page:
    """Rotate the dealer button, deal two hole cards per player, post blinds, and begin pre-flop play."""
    human = state.players[0]
    if human.chips <= 0:
        state.stage = "gameover"
        return game_over_page(state)

    n = len(state.players)
    for bot in state.players[1:]:
        if bot.chips <= 0:
            bot.chips = state.starting_stack
```

```
        state.hand_number += 1
        state.action_log = []
        state.message = ""
        state.deck = new_deck()
        state.community_cards = []
        state.pot = 0
        state.acted = [False] * n
        state.contributed = [0] * n
        state.min_raise = state.big_blind
        state.raises_this_street = 0
        state.raised_flags = [False] * n
        state.preflop_aggressor = -1
        state.dealer = (state.dealer + 1) % n

        for p in state.players:
            p.hole_cards = []
            p.current_bet = 0
            p.is_folded = False
        for _ in range(2):
            for p in state.players:
                p.hole_cards.append(state.deck.pop())

        small_idx = (state.dealer + 1) % n
        big_idx = (state.dealer + 2) % n
        post_blind(state, small_idx, state.small_blind, "small blind")
        post_blind(state, big_idx, state.big_blind, "big blind")

        state.stage = "preflop"
        state.current_turn_index = big_idx
        state.current_turn_index = next_actor(state)
        return trigger_bot_turns(state)


    @route
    def player_action(state: State, action_choice: str, raise_amount: str, auto_muck: bool) -> Page:
        """Validate and apply the human player's Fold, Check/Call, or Raise input from the table page."""
        state.auto_muck = auto_muck
        human = state.players[0]
        if state.stage not in BETTING_STAGES or state.current_turn_index != 0 or not can_act(human):
            return index(state)

        state.message = ""
        to_call = max(0, highest_bet(state) - human.current_bet)

        if action_choice == "Fold":
            apply_action(state, 0, "fold")
        elif action_choice == "Raise":
            try:
                raise_by = int(raise_amount.strip())
            except ValueError:
                state.message = "Raise amount must be a whole number of chips, for example " + str(state.min_raise) + "."
                return table_page(state)

            if raise_by >= state.min_raise and to_call + raise_by <= human.chips:
                apply_action(state, 0, "raise", raise_by)
            else:
                max_raise = human.chips - to_call
                if max_raise < state.min_raise:
                    state.message = ("You don't have enough chips to raise (minimum raise is "
                                     + str(state.min_raise) + "). Choose Check/Call or Fold.")
                else:
                    state.message = ("Invalid raise of " + str(raise_by) + ": enter between "
                                     + str(state.min_raise) + " and " + str(max_raise) + " chips.")
                return table_page(state)
        else:
            apply_action(state, 0, "call")

        state.current_turn_index = next_actor(state)
        return trigger_bot_turns(state)


    @route
    def trigger_bot_turns(state: State) -> Page:
```

```
        """Advance AI bot turns sequentially until the human must act, a street ends, or the hand finishes."""
        for _ in range(300):
            page = check_progress(state)
            if page is not None:
                return page
            idx = state.current_turn_index
            if not can_act(state.players[idx]):
                state.current_turn_index = next_actor(state)
                continue
            if idx == 0:
                return table_page(state)
            action, raise_by = choose_bot_action(state, idx)
            apply_action(state, idx, action, raise_by)
            state.current_turn_index = next_actor(state)
        return table_page(state)


    @route
    def advance_street(state: State) -> Page:
        """Reset street bets, burn a card, deal Flop/Turn/River community cards, and start betting."""
        cards_out = len(state.community_cards)
        if cards_out >= 5:
            return resolve_showdown(state)

        for p in state.players:
            p.current_bet = 0
        state.acted = [False] * len(state.players)
        state.min_raise = state.big_blind
        state.raises_this_street = 0

        state.deck.pop()
        if cards_out == 0:
            state.stage = "flop"
            for _ in range(3):
                state.community_cards.append(state.deck.pop())
        elif cards_out == 3:
            state.stage = "turn"
            state.community_cards.append(state.deck.pop())
        else:
            state.stage = "river"
            state.community_cards.append(state.deck.pop())

        state.current_turn_index = state.dealer
        state.current_turn_index = next_actor(state)
        return trigger_bot_turns(state)


    @route
    def reset_stack(state: State) -> Page:
        """Replenish the human player's stack to starting_stack after busting and deal a new hand."""
        human = state.players[0]
        if human.chips <= 0:
            human.chips = state.starting_stack
            state.rebuys += 1
            state.peak_chips = max(state.peak_chips, human.chips)
            state.message = ""
            return deal_hand(state)
        return index(state)


    @route
    def show_rules(state: State) -> Page:
        """Navigate to the Rules & Hand Rankings reference page."""
        return rules_page(state)


    @route
    def return_to_start(state: State) -> Page:
        """Return from an active or finished game back to the welcome/setup page."""
        state.stage = "setup"
        state.message = ""
        return welcome_page(state)
```

```
# ---------------------------------------------------------------------------
# Pages
# ---------------------------------------------------------------------------
def welcome_page(state: State) -> Page:
    """Build the initial game configuration and setup Page."""
    return Page(state, [
        Header("Texas Hold'em Poker", 1),
        "Play against four computer opponents. Set up your game below.",
        HorizontalRule(),
        "Your name:",
        TextBox("player_name", "User"),
        "Starting chip stack:",
        SelectBox("starting_stack", ["500", "1000", "2000", "5000"], "1000"),
        "Opponent difficulty:",
        SelectBox("difficulty", ["Easy", "Medium", "Hard"], "Medium"),
        "Blinds are set to 1/50 (big) and 1/100 (small) of your starting stack.",
        LineBreak(),
        Button("Start Game", start_new_game),
        Button("Rules & Hand Rankings", show_rules),
    ])


def table_page(state: State) -> Page:
    """Build the main active poker table Page with board graphics, action log, and betting controls."""
    human = state.players[0]
    to_call = max(0, highest_bet(state) - human.current_bet)
    max_raise = human.chips - to_call

    content = [Header("Hand #" + str(state.hand_number) + " - " + STAGE_LABELS.get(state.stage, state.stage), 2)]
    if state.message:
        content.append("⚠ " + state.message)
    content.append("Opponents: " + state.difficulty + "   |   Dealer, small blind and big blind rotate every hand")
    content.append(table_layout(state))

    if len(human.hole_cards) + len(state.community_cards) >= 5 and not human.is_folded:
        content.append("Your best hand right now: " + best_hand(human.hole_cards + state.community_cards)[2])

    if state.action_log:
        content.append(HorizontalRule())
        content.append("Recent action:")
        for line in state.action_log[-6:]:
            content.append("  • " + line)
    content.append(HorizontalRule())

    if state.current_turn_index == 0 and can_act(human):
        if to_call == 0:
            content.append("Your turn - nothing to call, you may check.")
        else:
            content.append("Your turn - " + str(min(to_call, human.chips)) + " to call.")
        if max_raise >= state.min_raise:
            content.append("Raise amount = chips added on top of the call. Minimum " + str(state.min_raise)
                           + ", maximum " + str(max_raise) + ".")
        else:
            content.append("You can't raise right now (not enough chips) - check/call or fold.")
        content.append("Action:")
        content.append(SelectBox("action_choice", ["Check/Call", "Raise", "Fold"], "Check/Call"))
        content.append("Raise amount (only used when Raise is selected):")
        content.append(TextBox("raise_amount", str(state.min_raise)))
        content.append("Auto-muck my losing hands at showdown:")
        content.append(CheckBox("auto_muck", state.auto_muck))
        content.append(LineBreak())
        content.append(Button("Submit Action", player_action))
    else:
        content.append(Button("Continue", trigger_bot_turns))
    content.append(Button("Rules & Hand Rankings", show_rules))
    content.append(Button("Quit to Start Screen", return_to_start))
    return Page(state, content)


def showdown_page(state: State) -> Page:
    """Build the post-hand showdown and payout summary Page."""
    content = [Header("Hand Results - Hand #" + str(state.hand_number), 1),
               Header(state.result_headline, 2),
               "Pot awarded: " + str(state.last_pot) + " chips",
```

```
                table_layout(state, reveal=state.reveal)]

        rows = [["Player", "Hole cards", "Hand", "Result", "Chips now"]]
        for i, p in enumerate(state.players):
            won = state.winnings[i] if i < len(state.winnings) else 0
            if p.is_folded:
                cards, hand = "Folded", "-"
            elif not state.reveal:
                cards = cards_text(p.hole_cards) if i == 0 else "Not shown"
                hand = "-"
            elif i == 0 and won == 0 and state.auto_muck:
                cards, hand = "Mucked (hidden)", "-"
            else:
                cards, hand = cards_text(p.hole_cards), state.hand_names[i]
            if won > 0:
                result = "★ WINNER  +" + str(won)
            elif state.contributed[i] > 0:
                result = "Lost " + str(state.contributed[i])
            else:
                result = "-"
            rows.append([p.name, cards, hand, result, str(p.chips)])
        content.append(Table(rows))

        if state.reveal and state.winning_cards:
            content.append("Winning 5-card hand: " + state.winning_hand_name)
            content.append(box(card_row(state.winning_cards, 62), bg=FELT, pad="8px", align="center"))
        elif not state.reveal:
            content.append("No showdown - the last player standing wins the pot.")

        content.append(HorizontalRule())
        if state.players[0].chips <= 0:
            content.append("You're out of chips.")
            content.append(Button("Continue", deal_hand))
        else:
            content.append(Button("Deal Next Hand", deal_hand))
        content.append(Button("Rules & Hand Rankings", show_rules))
        content.append(Button("Quit to Start Screen", return_to_start))
        return Page(state, content)


    def game_over_page(state: State) -> Page:
        """Build the Game Over and session statistics Page with rebuy and restart options."""
        played = state.hands_played
        rate = (str(round(100 * state.hands_won / played)) + "%") if played else "n/a"
        invested = state.starting_stack * (1 + state.rebuys)
        stats = [
            ["Session statistic", "Value"],
            ["Hands played", str(played)],
            ["Hands won", str(state.hands_won)],
            ["Win rate", rate],
            ["Biggest pot won", str(state.biggest_pot)],
            ["Peak chip count", str(state.peak_chips)],
            ["Re-buys used", str(state.rebuys)],
            ["Net result", str(state.players[0].chips - invested) + " chips"],
        ]
        return Page(state, [
            Header("Game Over", 1),
            "You're out of chips. You can re-buy and keep playing, or start over.",
            Table(stats),
            HorizontalRule(),
            Button("Re-buy " + str(state.starting_stack) + " chips and keep playing", reset_stack),
            HorizontalRule(),
            Header("Or start a brand-new game", 3),
            "Your name:",
            TextBox("player_name", state.players[0].name),
            "Starting chip stack:",
            SelectBox("starting_stack", ["500", "1000", "2000", "5000"], str(state.starting_stack)),
            "Opponent difficulty:",
            SelectBox("difficulty", ["Easy", "Medium", "Hard"], state.difficulty),
            Button("Start Over", start_new_game),
            Button("Rules & Hand Rankings", show_rules),
            Button("Back to Start Screen", return_to_start),
        ])
```

```
def rules_page(state: State) -> Page:
    """Build the Texas Hold'em Rules & Hand Rankings reference Page."""
    return Page(state, [
        Header("Rules & Hand Rankings", 1),
        Header("How a hand works", 3),
        "1. Each player is dealt two private hole cards. The small blind and big blind post forced bets.",
        "2. Pre-flop betting: starting left of the big blind, each player may check/call, raise, or fold.",
        "3. The flop (3 community cards) is dealt, followed by another betting round.",
        "4. The turn (1 card) is dealt, followed by betting.",
        "5. The river (1 card) is dealt, followed by the final betting round.",
        "6. Showdown: remaining players reveal their cards. The best five-card hand, made from any "
        "combination of your two hole cards and the five community cards, wins the pot.",
        "If everyone else folds, the last player wins the pot immediately without showing cards.",
        Header("Betting in this game", 3),
        "Check/Call: match the current bet (or check if there is none). "
        "Raise: type how many chips to add on top of the call - at least the minimum raise shown. "
        "Fold: give up your hand. If a player is all-in for less than the others bet, "
        "a side pot is created so they can only win what they matched.",
        "Auto-muck: tick the box to hide your hand at showdown when you lose.",
        Header("Hand rankings (best to worst)", 3),
        Table([["Rank", "Hand", "Description", "Example"]]
              + [[r[0], r[1], r[2], colorize(r[3])] for r in HAND_RANKINGS]),
        "When two players have the same hand type, the higher cards win; if all five cards play "
        "equally, the pot is split.",
        LineBreak(),
        Button("Back to the Game", index),
        Button("Back to Start Screen", return_to_start),
    ])


start_server(State())
```

User prompt: 1. make sure docstrings include: Function Purpose: A brief description of what the function does. 2. Parameters: Describe what the parameters are and what they represent. 3. Return Value: Describe what the return value is and what it represents. 4. Examples: Provide examples of how to use the function and what the expected output is.

Response: Here is the complete updated `FINAL PROJECT DRAFT_2.py` with every function docstring expanded to include **Function Purpose**, **Parameters**, **Return Value**, and **Examples**, while keeping zero non-constant global variables and full documentation of advanced techniques:

```
"""
Texas Hold'em Poker -- a Drafter website
========================================
Play one human against four computer opponents (5 players at the table).

Run:   pip install drafter
       python "FINAL PROJECT DRAFT_2.py"

Structure
---------
* Two dataclasses: Player and State (State holds players, community_cards,
  pot and current_turn_index, plus supporting fields the game needs).
* Nine routes: index, start_new_game, deal_hand, player_action,
  trigger_bot_turns, advance_street, reset_stack, show_rules, return_to_start.
* Five pages: welcome/setup, main table, showdown/hand results,
  game over/rebuy, rules & hand rankings.
"""
from dataclasses import dataclass, field
from itertools import combinations
from collections import Counter
from typing import List, Optional, Tuple
import random

# Direct imports from Drafter avoid module-level lowercase alias variables
from drafter import (
    Page, Header, HorizontalRule, LineBreak, TextBox, SelectBox, CheckBox,
    Button, Table, Span, Image, Div, route, start_server,
    change_color, change_background_color, change_padding,
    change_text_align, change_border,
)


# ---------------------------------------------------------------------------
```

```
# Constants (ALL_CAPS module-level constants only; no global variables)
# ---------------------------------------------------------------------------
RANKS = "23456789TJQKA"
SUITS = "♠♥♦♣"

RANK_NAMES = {2: "Two", 3: "Three", 4: "Four", 5: "Five", 6: "Six", 7: "Seven",
              8: "Eight", 9: "Nine", 10: "Ten", 11: "Jack", 12: "Queen",
              13: "King", 14: "Ace"}
RANK_PLURALS = {2: "Twos", 3: "Threes", 4: "Fours", 5: "Fives", 6: "Sixes",
                7: "Sevens", 8: "Eights", 9: "Nines", 10: "Tens", 11: "Jacks",
                12: "Queens", 13: "Kings", 14: "Aces"}

BETTING_STAGES = ("preflop", "flop", "turn", "river")
STAGE_LABELS = {"setup": "Setup", "preflop": "Pre-flop", "flop": "Flop",
                "turn": "Turn", "river": "River", "showdown": "Showdown",
                "gameover": "Game Over"}

NUM_PLAYERS = 5          # 1 human + 4 computer opponents

SUIT_COLORS = {"♦": "#ff8c00",    # diamonds - orange
               "♥": "#d40000",    # hearts   - red
               "♣": "#1b8a1b",    # clubs    - green
               "♠": "#0a1f6e"}    # spades   - dark blue

IMAGE_DIR = "images"
SUIT_LETTER = {"♠": "S", "♥": "H", "♦": "D", "♣": "C"}
FELT = "#0b5d34"          # table cloth
SEAT_BG = "#14703f"       # a player's seat
ACTING_BG = "#ffe27a"     # whose turn it is
WINNER_BG = "#ffd24a"     # winner of the hand
FOLDED_BG = "#4a5a52"     # folded players

ROLE_LABEL = {"dealer": "Dealer", "small_blind": "Small Blind", "big_blind": "Big Blind"}

# AI opponent behavior tuning parameters by difficulty level
BOT_PROFILES = {
    "Easy":   {"sims": 15, "noise": 0.12, "pre_noise": 1.5, "raise_k": 0.55, "fold_margin": 0.65,
               "bluff": 0.02, "cbet": 0.10, "trap": 0.00, "semi": 0.00, "reads": False,
               "read_floor": 0.0, "position": False, "pre_tight": -1.0, "raise_extra": 2.5,
               "limp_gap": 3.0, "call_loose": 2.0, "threebet": 13.0, "bluff3": 0.00},
    "Medium": {"sims": 45, "noise": 0.04, "pre_noise": 0.6, "raise_k": 0.45, "fold_margin": 0.95,
               "bluff": 0.06, "cbet": 0.40, "trap": 0.10, "semi": 0.04, "reads": True,
               "read_floor": 5.0, "position": True, "pre_tight": 0.0, "raise_extra": 0.5,
               "limp_gap": 1.5, "call_loose": 0.0, "threebet": 11.5, "bluff3": 0.00},
    "Hard":   {"sims": 100, "noise": 0.00, "pre_noise": 0.0, "raise_k": 0.38, "fold_margin": 1.05,
               "bluff": 0.10, "cbet": 0.55, "trap": 0.25, "semi": 0.08, "reads": True,
               "read_floor": 6.5, "position": True, "pre_tight": 0.0, "raise_extra": 0.0,
               "limp_gap": 1.0, "call_loose": 0.0, "threebet": 11.0, "bluff3": 0.06},
}

HAND_RANKINGS = [
    ["1", "Royal Flush", "A, K, Q, J, 10, all of the same suit", "A♠ K♠ Q♠ J♠ 10♠"],
    ["2", "Straight Flush", "Five cards in sequence, all of the same suit", "9♥ 8♥ 7♥ 6♥ 5♥"],
    ["3", "Four of a Kind", "Four cards of the same rank", "Q♠ Q♥ Q♦ Q♣ 4♠"],
    ["4", "Full House", "Three of a kind plus a pair", "J♠ J♥ J♦ 8♣ 8♠"],
    ["5", "Flush", "Any five cards of the same suit", "K♦ 10♦ 8♦ 5♦ 2♦"],
    ["6", "Straight", "Five cards in sequence, mixed suits (Ace is high or low)", "9♣ 8♦ 7♠ 6♥ 5♣"],
    ["7", "Three of a Kind", "Three cards of the same rank", "7♠ 7♥ 7♦ K♣ 2♠"],
    ["8", "Two Pair", "Two different pairs", "A♠ A♦ 5♥ 5♣ 9♠"],
    ["9", "One Pair", "Two cards of the same rank", "10♠ 10♥ A♦ 7♣ 3♠"],
    ["10", "High Card", "No combination - the highest card plays", "A♠ J♦ 8♣ 5♥ 3♠"],
]


# ---------------------------------------------------------------------------
# Dataclasses
# ---------------------------------------------------------------------------
@dataclass
class Player:
    """Represents one seat at the poker table (human or AI bot)."""
    name: str
    chips: int
    # ADVANCED TECHNIQUE (dataclasses.field): default_factory=list creates a brand-new
    # empty list per Player instance so instances do not share a single mutable list.
```

```
        hole_cards: List[str] = field(default_factory=list)
        current_bet: int = 0
        is_folded: bool = False


    @dataclass
    class State:
        """Holds the complete persistent application state passed between Drafter routes."""
        # --- Core table fields ---
        players: List[Player] = field(default_factory=list)
        community_cards: List[str] = field(default_factory=list)
        pot: int = 0
        current_turn_index: int = 0

        # --- Hand & betting progression fields ---
        stage: str = "setup"
        deck: List[str] = field(default_factory=list)
        dealer: int = 2
        hand_number: int = 0
        small_blind: int = 5
        big_blind: int = 10
        min_raise: int = 10
        raises_this_street: int = 0
        acted: List[bool] = field(default_factory=list)
        contributed: List[int] = field(default_factory=list)
        starting_stack: int = 1000
        difficulty: str = "Medium"
        auto_muck: bool = False
        message: str = ""
        action_log: List[str] = field(default_factory=list)

        # --- Opponent modeling data tracked for the AI ---
        raised_flags: List[bool] = field(default_factory=list)
        preflop_aggressor: int = -1
        opp_raises: List[int] = field(default_factory=list)
        opp_actions: List[int] = field(default_factory=list)

        # --- Showdown & hand result fields ---
        result_headline: str = ""
        reveal: bool = False
        hand_names: List[str] = field(default_factory=list)
        winnings: List[int] = field(default_factory=list)
        winning_cards: List[str] = field(default_factory=list)
        winning_hand_name: str = ""
        last_pot: int = 0

        # --- Session statistics ---
        hands_played: int = 0
        hands_won: int = 0
        biggest_pot: int = 0
        rebuys: int = 0
        peak_chips: int = 0


    # ------------------------------------------------------------------------
    # Cards, text formatting, and image/layout helpers
    # ------------------------------------------------------------------------
    def new_deck() -> List[str]:
        """
        Function Purpose:
            Build and randomly shuffle a standard 52-card deck.
        Parameters:
            None.
        Return Value:
            List[str]: A list of 52 shuffled two-character card strings (e.g., 'A♠', 'TH').
        Examples:
            >>> d = new_deck()
            >>> len(d)
            52
            >>> "A♠" in d
            True
        """
        # Nested list comprehension generates all 13x4 = 52 rank-suit combinations
        deck = [r + s for r in RANKS for s in SUITS]
```

```
    random.shuffle(deck)
    return deck


def show_card(card: str) -> str:
    """
    Function Purpose:
        Convert the internal single-character ten code ('T') into '10' for readable display.
    Parameters:
        card (str): A two-character card string such as 'T♠' or 'K♦'.
    Return Value:
        str: The formatted card string with 'T' replaced by '10'.
    Examples:
        >>> show_card("T♠")
        '10♠'
        >>> show_card("A♥")
        'A♥'
    """
    return card.replace("T", "10")


def styled(text: str, color: str):
    """
    Function Purpose:
        Wrap a text string in a colored Drafter Span component.
    Parameters:
        text (str): The text content to display.
        color (str): A CSS color name or hex code (e.g., '#d40000').
    Return Value:
        Span: A Drafter Span component styled with the requested text color.
    Examples:
        >>> comp = styled("[A♥]", "#d40000")
        >>> type(comp).__name__
        'Span'
    """
    return change_color(Span(text), color)


def rich(*parts):
    """
    Function Purpose:
        Combine plain strings and styled Span components into a single inline element.
    Parameters:
        *parts (Union[str, Span]): Variable number of strings or Drafter Span objects to join.
    Return Value:
        Union[str, Span]: A concatenated string if all parts are strings, otherwise a combined Span.
    Examples:
        >>> rich("Hello", " ", "World")
        'Hello World'
        >>> type(rich(styled("[A♠]", "#0a1f6e"), " wins")).__name__
        'Span'
    """
    # ADVANCED TECHNIQUE (*args variadic parameters & *parts unpacking): accepts any
    # number of positional arguments as a tuple and unpacks them into Span(*parts).
    if all(isinstance(p, str) for p in parts):
        return "".join(parts)
    return Span(*parts)


def card_piece(card: str):
    """
    Function Purpose:
        Render a single card string as a suit-colored bracketed text token.
    Parameters:
        card (str): A two-character card string (e.g., 'Q♣').
    Return Value:
        Span: A colored Span displaying the bracketed card, such as '[Q♣]'.
    Examples:
        >>> piece = card_piece("T♦")
        >>> type(piece).__name__
        'Span'
    """
    return styled("[" + show_card(card) + "]", SUIT_COLORS[card[1]])
```

```
def colorize(text: str):
    """
    Function Purpose:
        Color every card token (such as 'A♠' or '10♦') inside a space-separated string.
    Parameters:
        text (str): A string containing plain words and/or card tokens separated by spaces.
    Return Value:
        Union[str, Span]: A rich text component with each card token colored by its suit.
    Examples:
        >>> type(colorize("A♠ K♠ Q♠ J♠ 10♠")).__name__
        'Span'
        >>> colorize("No cards here")
        'No cards here'
    """
    parts = []
    for i, token in enumerate(text.split(" ")):
        if i:
            parts.append(" ")
        if token and token[-1] in SUIT_COLORS:
            parts.append(styled(token, SUIT_COLORS[token[-1]]))
        else:
            parts.append(token)
    return rich(*parts)


def cards_text(cards: List[str]):
    """
    Function Purpose:
        Render a list of cards as suit-colored text tokens separated by spaces.
    Parameters:
        cards (List[str]): A list of two-character card strings (e.g., ['A♠', 'K♥']).
    Return Value:
        Union[str, Span]: A combined Span showing each card in brackets colored by suit.
    Examples:
        >>> type(cards_text(["A♠", "K♥"])).__name__
        'Span'
        >>> cards_text([])
        ''
    """
    parts = []
    for i, c in enumerate(cards):
        if i:
            parts.append(" ")
        parts.append(card_piece(c))
    return rich(*parts)


def board_text(cards: List[str]):
    """
    Function Purpose:
        Render up to 5 community cards as colored text with '[  ]' placeholders for undealt slots.
    Parameters:
        cards (List[str]): Currently dealt community cards (0 to 5 card strings).
    Return Value:
        Union[str, Span]: A 5-slot display combining dealt card pieces and empty placeholders.
    Examples:
        >>> board_text([])
        '[  ] [  ] [  ] [  ] [  ]'
        >>> type(board_text(["A♠", "K♦", "7♣"])).__name__
        'Span'
    """
    parts = []
    for i in range(5):
        if i:
            parts.append(" ")
        parts.append(card_piece(cards[i]) if i < len(cards) else "[  ]")
    return rich(*parts)


def card_file(card: str) -> str:
    """
    Function Purpose:
        Map a card code like 'A♠' to its relative PNG image file path.
```

```
    Parameters:
        card (str): A two-character card string consisting of rank and suit symbol.
    Return Value:
        str: Relative path to the corresponding PNG file (e.g., 'images/cards/AS.png').
    Examples:
        >>> card_file("A♠")
        'images/cards/AS.png'
        >>> card_file("T♥")
        'images/cards/TH.png'
    """
    return IMAGE_DIR + "/cards/" + card[0] + SUIT_LETTER[card[1]] + ".png"


def card_image(card: str, width: int = 62):
    """
    Function Purpose:
        Create a Drafter Image component displaying the face of a given playing card.
    Parameters:
        card (str): A two-character card string (e.g., 'J♦').
        width (int): Desired display width of the image in pixels (default 62).
    Return Value:
        Image: A Drafter Image component pointing to the card's PNG asset.
    Examples:
        >>> img = card_image("J♦", width=70)
        >>> type(img).__name__
        'Image'
    """
    return Image(card_file(card), width=width)


def back_image(width: int = 62):
    """
    Function Purpose:
        Create a Drafter Image component representing the back of a face-down playing card.
    Parameters:
        width (int): Desired display width of the image in pixels (default 62).
    Return Value:
        Image: A Drafter Image component pointing to 'images/back.png'.
    Examples:
        >>> img = back_image(62)
        >>> type(img).__name__
        'Image'
    """
    return Image(IMAGE_DIR + "/back.png", width=width)


def empty_image(width: int = 62):
    """
    Function Purpose:
        Create a Drafter Image component representing an empty placeholder card slot.
    Parameters:
        width (int): Desired display width of the image in pixels (default 62).
    Return Value:
        Image: A Drafter Image component pointing to 'images/empty.png'.
    Examples:
        >>> img = empty_image(70)
        >>> type(img).__name__
        'Image'
    """
    return Image(IMAGE_DIR + "/empty.png", width=width)


def badge_image(kind: str, width: int = 30):
    """
    Function Purpose:
        Create a Drafter Image component for a table position badge (dealer or blinds).
    Parameters:
        kind (str): Badge identifier: 'dealer', 'small_blind', or 'big_blind'.
        width (int): Desired display width of the badge in pixels (default 30).
    Return Value:
        Image: A Drafter Image component pointing to 'images/badges/<kind>.png'.
    Examples:
        >>> img = badge_image("dealer")
        >>> type(img).__name__
```

```
        'Image'
    """
    return Image(IMAGE_DIR + "/badges/" + kind + ".png", width=width)


def decorate(component, bg=None, color=None, pad=None, align=None, border=None):
    """
    Function Purpose:
        Apply optional CSS styling attributes (background, text color, padding, alignment, border)
        to a Drafter component.
    Parameters:
        component: Any Drafter UI component to style.
        bg (Optional[str]): Background color string (e.g., '#0b5d34').
        color (Optional[str]): Foreground text color string (e.g., 'white').
        pad (Optional[str]): CSS padding value (e.g., '8px').
        align (Optional[str]): CSS text-align value (e.g., 'center').
        border (Optional[str]): CSS border specification (e.g., '2px solid #e8d9a8').
    Return Value:
        Component: The styled Drafter component.
    Examples:
        >>> styled_div = decorate(Div("Hello"), bg="#0b5d34", color="white", pad="8px")
        >>> type(styled_div).__name__
        'Div'
    """
    try:
        if bg:
            component = change_background_color(component, bg)
        if color:
            component = change_color(component, color)
        if pad:
            component = change_padding(component, pad)
        if align:
            component = change_text_align(component, align)
        if border:
            component = change_border(component, border)
    except Exception:
        pass
    return component


def box(items, **style):
    """
    Function Purpose:
        Wrap a list of Drafter components inside a styled Div container.
    Parameters:
        items (list): Child Drafter components or strings to place inside the Div.
        **style: Keyword style arguments (bg, color, pad, align, border) passed to decorate().
    Return Value:
        Div: A styled Drafter Div container holding all given items.
    Examples:
        >>> container = box(["Pot: 100 chips"], bg="#0b5d34", color="white", align="center")
        >>> type(container).__name__
        'Div'
    """
    # ADVANCED TECHNIQUE (**kwargs keyword unpacking): collects arbitrary style keyword
    # arguments (bg, color, pad, align, border) into a dict and forwards them to decorate().
    return decorate(Div(*items), **style)


def card_row(cards: List[str], width: int = 62, hidden: bool = False, slots: int = 0) -> list:
    """
    Function Purpose:
        Build a horizontal list of card Image components (face-up, face-down, or empty placeholders).
    Parameters:
        cards (List[str]): Card codes to render in the row.
        width (int): Width in pixels for each card image (default 62).
        hidden (bool): If True, render card backs instead of card faces (default False).
        slots (int): Total slot count; pads remaining positions with empty card slots (default 0).
    Return Value:
        list: A list of Image components and space separators ready for rendering.
    Examples:
        >>> row = card_row(["A♠", "K♠"], width=62, hidden=False)
        >>> len(row)
        3
```

```
        >>> hidden_row = card_row(["A♠", "K♠"], hidden=True)
        >>> len(hidden_row)
        3
    """
    row = []
    for c in cards:
        if row:
            row.append(" ")
        row.append(back_image(width) if hidden else card_image(c, width))
    for _ in range(slots - len(cards)):
        if row:
            row.append(" ")
        row.append(empty_image(width))
    return row


def seat_roles(state: State, i: int) -> List[str]:
    """
    Function Purpose:
        Determine which rotating table roles ('dealer', 'small_blind', 'big_blind') belong to seat i.
    Parameters:
        state (State): The current game state containing dealer index and players list.
        i (int): Zero-based seat index of the player to check.
    Return Value:
        List[str]: Zero or more role keys assigned to seat i for the current hand.
    Examples:
        >>> s = State(players=[Player("A", 100), Player("B", 100), Player("C", 100)], dealer=0)
        >>> seat_roles(s, 0)
        ['dealer']
        >>> seat_roles(s, 1)
        ['small_blind']
    """
    n = len(state.players)
    roles = []
    if i == state.dealer:
        roles.append("dealer")
    if i == (state.dealer + 1) % n:
        roles.append("small_blind")
    if i == (state.dealer + 2) % n:
        roles.append("big_blind")
    return roles


def seat_box(state: State, i: int, reveal: bool = False):
    """
    Function Purpose:
        Build the styled visual seat box for player i, showing badges, chip stack, bet, status, and cards.
    Parameters:
        state (State): The current game state.
        i (int): Zero-based index of the player in state.players.
        reveal (bool): Whether opponents' non-folded hole cards should be shown face-up (default False).
    Return Value:
        Div: A styled Drafter Div representing player i's seat at the table.
    Examples:
        >>> s = State(players=[Player("User", 1000)], stage="preflop", current_turn_index=0)
        >>> type(seat_box(s, 0)).__name__
        'Div'
    """
    p = state.players[i]
    in_betting = state.stage in BETTING_STAGES
    won = state.winnings[i] if (state.stage == "showdown" and i < len(state.winnings)) else 0
    acting = in_betting and i == state.current_turn_index

    items = []
    for role in seat_roles(state, i):
        items.append(badge_image(role))
        items.append(" ")
    items.append(p.name + (" (you)" if i == 0 else ""))
    items.append(LineBreak())
    roles = [ROLE_LABEL[r] for r in seat_roles(state, i)]
    if roles:
        items.append(" / ".join(roles))
        items.append(LineBreak())
    items.append("Chips: " + str(p.chips) + (("   Bet: " + str(p.current_bet)) if in_betting else ""))
```

```
        items.append(LineBreak())

        if p.is_folded:
            status = "Folded"
        elif won > 0:
            status = "★ WINNER +" + str(won)
        elif acting:
            status = "▶ Your turn" if i == 0 else "▶ Acting"
        elif p.chips == 0 and in_betting:
            status = "All-in"
        else:
            status = "In hand" if in_betting else ""
        if status:
            items.append(status)
            items.append(LineBreak())

        if p.hole_cards and not p.is_folded:
            mucked = reveal and i == 0 and won == 0 and state.auto_muck
            face_up = (i == 0 and not mucked) or (reveal and i != 0)
            items.extend(card_row(p.hole_cards, 62, hidden=not face_up))

        if p.is_folded:
            bg, fg = FOLDED_BG, "#d0d8d4"
        elif won > 0:
            bg, fg = WINNER_BG, "#1a1a1a"
        elif acting:
            bg, fg = ACTING_BG, "#1a1a1a"
        else:
            bg, fg = SEAT_BG, "white"
        return box(items, bg=bg, color=fg, pad="8px", align="center", border="2px solid #e8d9a8")


    def table_layout(state: State, reveal: bool = False):
        """
        Function Purpose:
            Arrange all 5 player seats, the pot/blinds readout, and the 5-card community board
            into a 3x3 green-felt poker table layout.
        Parameters:
            state (State): The current game state containing all 5 players and community cards.
            reveal (bool): Whether to reveal all active players' hole cards (used at showdown).
        Return Value:
            Table: A styled Drafter Table representing the entire poker table.
        Examples:
            >>> s = State(players=[Player("P" + str(k), 1000) for k in range(5)], stage="preflop")
            >>> type(table_layout(s)).__name__
            'Table'
        """
        pot_box = box([
            "Pot: " + str(state.pot if state.stage in BETTING_STAGES else state.last_pot) + " chips",
            LineBreak(),
            STAGE_LABELS.get(state.stage, state.stage) + "  -  Blinds " + str(state.small_blind) + "/" + str(state.big_blind
        ], bg=FELT, color="white", pad="6px", align="center")

        middle = card_row(state.community_cards, 70, slots=5)
        board_box = box(middle, bg=FELT, color="white", pad="8px", align="center")
        blank = box([" "], bg=FELT)
        seats = {i: seat_box(state, i, reveal) for i in range(len(state.players))}
        rows = [
            [seats[2], pot_box, seats[3]],
            [seats[1], board_box, seats[4]],
            [blank, seats[0], blank],
        ]
        return decorate(Table(rows), bg=FELT, pad="10px", align="center")


    # ------------------------------------------------------------------------
    # Hand evaluation and Monte Carlo equity estimation
    # ------------------------------------------------------------------------
    def score_five(cards) -> Tuple[int, ...]:
        """
        Function Purpose:
            Evaluate an exact 5-card poker hand and return a lexicographically comparable score tuple.
        Parameters:
            cards (Sequence[str]): Exactly 5 two-character card strings (e.g., ['A♠', 'K♠', 'Q♠', 'J♠', 'T♠']).
```

```
        Return Value:
            Tuple[int, ...]: Tuple starting with category rank (0=High Card .. 9=Royal Flush)
                followed by tiebreaker card ranks in descending significance.
        Examples:
            >>> score_five(["A♠", "K♠", "Q♠", "J♠", "T♠"])
            (9, 14)
            >>> score_five(["7♠", "7♥", "7♦", "K♣", "2♠"])
            (3, 7, 13, 2)
        """
        # Convert rank characters ('2'..'A') to integers 2..14 sorted high-to-low
        ranks = sorted((RANKS.index(c[0]) + 2 for c in cards), reverse=True)
        # Set comprehension checks whether all 5 cards share a single suit
        flush = len({c[1] for c in cards}) == 1
        # ADVANCED TECHNIQUE (collections.Counter + lambda key): Counter tallies rank
        # frequencies; sorting by (count, rank) groups pairs/trips/quads ahead of kickers.
        counts = Counter(ranks)
        groups = sorted(counts.items(), key=lambda kv: (kv[1], kv[0]), reverse=True)
        shape = [n for _, n in groups]
        ordered = [r for r, _ in groups]

        straight_high = 0
        if len(counts) == 5:
            if ranks[0] - ranks[4] == 4:
                straight_high = ranks[0]
            elif ranks == [14, 5, 4, 3, 2]:        # Ace-low straight ("wheel": A-2-3-4-5)
                straight_high = 5

        # ADVANCED TECHNIQUE (tuple comparison & * unpacking): Python compares tuples
        # lexicographically element-by-element, so higher hand tiers and higher kickers win.
        if straight_high and flush:
            return (9 if straight_high == 14 else 8, straight_high)
        if shape == [4, 1]:
            return (7, *ordered)
        if shape == [3, 2]:
            return (6, *ordered)
        if flush:
            return (5, *ranks)
        if straight_high:
            return (4, straight_high)
        if shape == [3, 1, 1]:
            return (3, *ordered)
        if shape == [2, 2, 1]:
            return (2, *ordered)
        if shape == [2, 1, 1, 1]:
            return (1, *ordered)
        return (0, *ranks)


    def best_score(cards) -> Tuple[int, ...]:
        """
        Function Purpose:
            Find the highest possible 5-card hand score from 5 to 7 available cards.
        Parameters:
            cards (Sequence[str]): 5, 6, or 7 two-character card strings.
        Return Value:
            Tuple[int, ...]: The maximum score_five() tuple across all 5-card combinations.
        Examples:
            >>> best_score(["A♠", "A♦", "K♠", "Q♠", "J♠", "T♠", "2♣"])
            (9, 14)
        """
        best = None
        # ADVANCED TECHNIQUE (itertools.combinations): generates all 5-card subsets
        # from up to 7 available cards (7 choose 5 = 21 combinations) without manual loops.
        for five in combinations(cards, 5):
            s = score_five(five)
            if best is None or s > best:
                best = s
        return best


    def describe_hand(score: Tuple[int, ...]) -> str:
        """
        Function Purpose:
            Convert a numerical hand score tuple into a human-readable poker hand name.
```

```
        Parameters:
            score (Tuple[int, ...]): A score tuple produced by score_five() or best_score().
        Return Value:
            str: Descriptive name of the hand (e.g., 'Two Pair, Aces and Fives').
        Examples:
            >>> describe_hand((9, 14))
            'Royal Flush'
            >>> describe_hand((2, 14, 5, 9))
            'Two Pair, Aces and Fives'
        """
        cat = score[0]
        if cat == 9:
            return "Royal Flush"
        if cat == 8:
            return "Straight Flush, " + RANK_NAMES[score[1]] + " high"
        if cat == 7:
            return "Four of a Kind, " + RANK_PLURALS[score[1]]
        if cat == 6:
            return "Full House, " + RANK_PLURALS[score[1]] + " full of " + RANK_PLURALS[score[2]]
        if cat == 5:
            return "Flush, " + RANK_NAMES[score[1]] + " high"
        if cat == 4:
            return "Straight, " + RANK_NAMES[score[1]] + " high"
        if cat == 3:
            return "Three of a Kind, " + RANK_PLURALS[score[1]]
        if cat == 2:
            return "Two Pair, " + RANK_PLURALS[score[1]] + " and " + RANK_PLURALS[score[2]]
        if cat == 1:
            return "Pair of " + RANK_PLURALS[score[1]]
        return "High Card, " + RANK_NAMES[score[1]]


    def best_hand(cards) -> Tuple[Tuple[int, ...], List[str], str]:
        """
        Function Purpose:
            Determine the best 5-card hand from 5 to 7 cards and return its score, ordered cards, and name.
        Parameters:
            cards (Sequence[str]): 5 to 7 two-character card strings (hole cards + community cards).
        Return Value:
            Tuple[Tuple[int, ...], List[str], str]: (score_tuple, ordered_best_5_cards, hand_name).
        Examples:
            >>> sc, five, name = best_hand(["A♠", "K♠", "Q♠", "J♠", "T♠", "2♦", "3♣"])
            >>> name
            'Royal Flush'
            >>> len(five)
            5
        """
        best = None
        best_five = None
        for five in combinations(cards, 5):
            s = score_five(five)
            if best is None or s > best:
                best, best_five = s, list(five)
        counts = Counter(c[0] for c in best_five)
        best_five.sort(key=lambda c: (counts[c[0]], RANKS.index(c[0])), reverse=True)
        if best[0] in (4, 8) and best[1] == 5:     # place Ace at the end for A-2-3-4-5 straights
            best_five.append(best_five.pop(0))
        return best, best_five, describe_hand(best)


    def chen_score(cards) -> float:
        """
        Function Purpose:
            Calculate the Chen Formula pre-flop starting hand strength rating for two hole cards.
        Parameters:
            cards (Sequence[str]): Two two-character hole card strings (e.g., ['A♠', 'A♥']).
        Return Value:
            float: Pre-flop hand strength score ranging from roughly -1.0 (weakest) to 20.0 (pocket Aces).
        Examples:
            >>> chen_score(["A♠", "A♥"])
            20.0
            >>> chen_score(["7♦", "2♣"])
            -1.5
        """
```

```
        r1, r2 = sorted((RANKS.index(c[0]) + 2 for c in cards), reverse=True)
        top = {14: 10.0, 13: 8.0, 12: 7.0, 11: 6.0}.get(r1, r1 / 2.0)
        if r1 == r2:
            return max(5.0, top * 2)
        score = top
        if cards[0][1] == cards[1][1]:
            score += 2                                  # suited bonus
        gap = r1 - r2 - 1
        score -= {0: 0, 1: 1, 2: 2, 3: 4}.get(gap, 5)
        if gap <= 1 and r1 < 12:
            score += 1                                  # connectedness bonus below Queen
        return score


    def estimate_equity(hole: List[str], board: List[str], floors: List[float], sims: int) -> float:
        """
        Function Purpose:
            Estimate a hand's probability of winning at showdown via Monte Carlo simulation.
        Parameters:
            hole (List[str]): The bot's 2 private hole cards.
            board (List[str]): Currently dealt community cards (3 to 5 cards).
            floors (List[float]): Minimum Chen score assumed for each remaining opponent (0.0 = any hand).
            sims (int): Number of random board/opponent trials to simulate.
        Return Value:
            float: Estimated win equity between 0.0 (0% chance) and 1.0 (100% chance).
        Examples:
            >>> eq = estimate_equity(["A♠", "A♥"], ["A♦", "A♣", "K♠", "Q♠", "J♠"], [0.0], 20)
            >>> eq
            1.0
        """
        # ADVANCED TECHNIQUE (Monte Carlo simulation & set difference): builds the unseen deck
        # via set membership and samples `sims` random board completions and opponent holdings.
        seen = set(hole) | set(board)
        deck = [r + s for r in RANKS for s in SUITS if (r + s) not in seen]
        need = 5 - len(board)
        wins = 0.0
        for _ in range(sims):
            d = deck[:]
            random.shuffle(d)
            full_board = board + d[:need]
            rest = d[need:]
            mine = best_score(hole + full_board)
            best_opp = None
            for floor in floors:
                picked = None
                if floor > 0:
                    for _try in range(12):
                        i, j = random.sample(range(len(rest)), 2)
                        if chen_score((rest[i], rest[j])) >= floor:
                            picked = (i, j)
                            break
                if picked is None:
                    picked = (len(rest) - 1, len(rest) - 2)
                hand = [rest[picked[0]], rest[picked[1]]]
                for k in sorted(picked, reverse=True):
                    rest.pop(k)
                sc = best_score(hand + full_board)
                if best_opp is None or sc > best_opp:
                    best_opp = sc
            if mine > best_opp:
                wins += 1
            elif mine == best_opp:
                wins += 0.5
        return wins / sims


    # ------------------------------------------------------------------------
    # Betting-round helpers
    # ------------------------------------------------------------------------
    def can_act(player: Player) -> bool:
        """
        Function Purpose:
            Check whether a player is eligible to make a betting action (not folded and has chips left).
        Parameters:
```

```
        player (Player): The Player instance to check.
    Return Value:
        bool: True if the player is still in the hand and has chips > 0, False otherwise.
    Examples:
        >>> can_act(Player("User", 500, is_folded=False))
        True
        >>> can_act(Player("User", 0, is_folded=False))
        False
    """
    return (not player.is_folded) and player.chips > 0


def highest_bet(state: State) -> int:
    """
    Function Purpose:
        Find the largest current_bet among all non-folded players in the current betting round.
    Parameters:
        state (State): The current game state.
    Return Value:
        int: The highest active bet amount that players must match to stay in the hand.
    Examples:
        >>> s = State(players=[Player("A", 90, current_bet=10), Player("B", 80, current_bet=20)])
        >>> highest_bet(s)
        20
    """
    top = 0
    for p in state.players:
        if not p.is_folded and p.current_bet > top:
            top = p.current_bet
    return top


def next_actor(state: State) -> int:
    """
    Function Purpose:
        Find the seat index of the next player clockwise who is able to act.
    Parameters:
        state (State): The current game state containing players and current_turn_index.
    Return Value:
        int: Seat index of the next player who can act (or current_turn_index if none can act).
    Examples:
        >>> s = State(players=[Player("A", 100), Player("B", 100, is_folded=True), Player("C", 100)], current_turn_index
        >>> next_actor(s)
        2
    """
    n = len(state.players)
    for step in range(1, n + 1):
        candidate = (state.current_turn_index + step) % n
        if can_act(state.players[candidate]):
            return candidate
    return state.current_turn_index


def betting_round_complete(state: State) -> bool:
    """
    Function Purpose:
        Determine whether the current betting round is finished so play can advance to the next street.
    Parameters:
        state (State): The current game state.
    Return Value:
        bool: True if every active player with chips has matched the highest bet and had a turn to act.
    Examples:
        >>> s = State(players=[Player("A", 90, current_bet=10), Player("B", 90, current_bet=10)], acted=[True, True])
        >>> betting_round_complete(s)
        True
    """
    highest = highest_bet(state)
    able = 0
    waiting = 0
    for i, p in enumerate(state.players):
        if p.is_folded or p.chips == 0:
            continue
        if p.current_bet != highest:
            return False
```

```
            able += 1
            if not state.acted[i]:
                waiting += 1
        if able >= 2 and waiting > 0:
            return False
        return True


    def add_log(state: State, text: str) -> None:
        """
        Function Purpose:
            Append a formatted action message to the current hand's action_log list.
        Parameters:
            state (State): The current game state.
            text (str): Description of the action taken (e.g., 'Bot 1 calls 10').
        Return Value:
            None.
        Examples:
            >>> s = State()
            >>> add_log(s, "User checks")
            >>> s.action_log
            ['User checks']
        """
        state.action_log.append(text)


    def put_chips(state: State, idx: int, amount: int) -> None:
        """
        Function Purpose:
            Transfer up to `amount` chips from player idx's stack into their current_bet, pot, and contributed total.
        Parameters:
            state (State): The current game state.
            idx (int): Seat index of the player putting chips in.
            amount (int): Desired number of chips to commit (capped at the player's remaining chips).
        Return Value:
            None.
        Examples:
            >>> s = State(players=[Player("User", 100)], contributed=[0])
            >>> put_chips(s, 0, 25)
            >>> (s.players[0].chips, s.pot)
            (75, 25)
        """
        p = state.players[idx]
        amount = min(amount, p.chips)
        p.chips -= amount
        p.current_bet += amount
        state.pot += amount
        state.contributed[idx] += amount


    def post_blind(state: State, idx: int, amount: int, label: str) -> None:
        """
        Function Purpose:
            Post a forced small or big blind for player idx and log the blind posting.
        Parameters:
            state (State): The current game state.
            idx (int): Seat index of the player posting the blind.
            amount (int): Blind chip amount to post.
            label (str): Blind label for the log ('small blind' or 'big blind').
        Return Value:
            None.
        Examples:
            >>> s = State(players=[Player("User", 100)], contributed=[0])
            >>> post_blind(s, 0, 5, "small blind")
            >>> s.pot
            5
        """
        put_chips(state, idx, amount)
        p = state.players[idx]
        add_log(state, p.name + " posts the " + label + " (" + str(p.current_bet) + ")")


    def apply_action(state: State, idx: int, action: str, raise_by: int = 0) -> None:
        """
```

```
    Function Purpose:
        Execute a player's 'fold', 'call' (check/call), or 'raise' decision and update turn flags.
    Parameters:
        state (State): The current game state.
        idx (int): Seat index of the acting player.
        action (str): One of 'fold', 'call', or 'raise'.
        raise_by (int): Additional chips raised on top of the call amount when action == 'raise'.
    Return Value:
        None.
    Examples:
        >>> s = State(players=[Player("User", 100)], acted=[False], contributed=[0], opp_actions=[0], opp_raises=[0], ra
        >>> apply_action(s, 0, "call")
        >>> s.action_log[-1]
        'User checks'
    """
    p = state.players[idx]
    to_call = max(0, highest_bet(state) - p.current_bet)
    state.opp_actions[idx] += 1
    if action == "fold":
        p.is_folded = True
        add_log(state, p.name + " folds")
    elif action == "raise":
        put_chips(state, idx, to_call + raise_by)
        state.min_raise = max(state.min_raise, raise_by)
        state.raises_this_street += 1
        state.opp_raises[idx] += 1
        state.raised_flags[idx] = True
        if state.stage == "preflop":
            state.preflop_aggressor = idx
        for j in range(len(state.players)):
            if j != idx:
                state.acted[j] = False
        note = " (all-in)" if p.chips == 0 else ""
        add_log(state, p.name + " raises to " + str(p.current_bet) + note)
    else:
        paid = min(to_call, p.chips)
        put_chips(state, idx, paid)
        if to_call == 0:
            add_log(state, p.name + " checks")
        else:
            note = " (all-in)" if p.chips == 0 else ""
            add_log(state, p.name + " calls " + str(paid) + note)
    state.acted[idx] = True


# ------------------------------------------------------------------------
# Computer opponent decision making
# ------------------------------------------------------------------------
def raise_rate(state: State, j: int) -> float:
    """
    Function Purpose:
        Compute how frequently player j has raised across the session (defaults to 0.25 until 8 actions).
    Parameters:
        state (State): The current game state containing opp_actions and opp_raises.
        j (int): Seat index of the opponent being evaluated.
    Return Value:
        float: Ratio of raises to total actions (between 0.0 and 1.0), or 0.25 if sample size < 8.
    Examples:
        >>> s = State(opp_actions=[10], opp_raises=[4])
        >>> raise_rate(s, 0)
        0.4
    """
    actions = state.opp_actions[j]
    return state.opp_raises[j] / actions if actions >= 8 else 0.25


def bot_context(state: State, idx: int, profile: dict) -> dict:
    """
    Function Purpose:
        Assemble table position, call cost, stack room, opponent reads, and raise legality for bot idx.
    Parameters:
        state (State): The current game state.
        idx (int): Seat index of the acting bot.
        profile (dict): Difficulty parameter dictionary from BOT_PROFILES.
```

```
        Return Value:
            dict: Context metrics ('highest', 'to_call', 'room', 'opps', 'lateness', 'agg', 'agg_scale', 'can_raise').
        Examples:
            >>> s = State(players=[Player("U", 100), Player("B1", 100)], dealer=0, min_raise=10)
            >>> ctx = bot_context(s, 1, BOT_PROFILES["Medium"])
            >>> ctx["to_call"]
            0
        """
        bot = state.players[idx]
        n = len(state.players)
        highest = highest_bet(state)
        to_call = max(0, highest - bot.current_bet)
        room = bot.chips - to_call
        opps = [j for j, p in enumerate(state.players) if j != idx and not p.is_folded]

        lateness = ((idx - state.dealer - 1) % n) / (n - 1) if profile["position"] else 0.5

        agg, agg_scale = 0.0, 1.0
        if profile["reads"] and to_call > 0:
            aggressor = max(opps, key=lambda j: state.players[j].current_bet)
            rate = raise_rate(state, aggressor)
            if rate < 0.15:
                agg, agg_scale = 1.5, 1.15
            elif rate > 0.40:
                agg, agg_scale = -1.0, 0.92

        return {"highest": highest, "to_call": to_call, "room": room, "opps": opps,
                "lateness": lateness, "agg": agg, "agg_scale": agg_scale,
                "can_raise": room >= state.min_raise and state.raises_this_street < 3}


    def make_raise(state: State, ctx: dict, raise_by: int) -> Tuple[str, int]:
        """
        Function Purpose:
            Clamp a desired raise increment to legal min_raise and available chip room, or fall back to calling.
        Parameters:
            state (State): The current game state containing min_raise.
            ctx (dict): Bot context dictionary from bot_context().
            raise_by (int): Desired raise increment above the call amount.
        Return Value:
            Tuple[str, int]: ('raise', legal_raise_by) if raising is allowed, otherwise ('call', 0).
        Examples:
            >>> s = State(min_raise=10)
            >>> make_raise(s, {"can_raise": True, "room": 50}, 25)
            ('raise', 25)
        """
        if not ctx["can_raise"]:
            return ("call", 0)
        raise_by = max(int(raise_by), state.min_raise)
        return ("raise", min(raise_by, ctx["room"]))


    def pot_raise(state: State, ctx: dict, fraction: float) -> Tuple[str, int]:
        """
        Function Purpose:
            Compute a bet or raise sized to a given fraction of the pot, rounded down to the small blind.
        Parameters:
            state (State): The current game state containing pot and small_blind.
            ctx (dict): Bot context dictionary from bot_context().
            fraction (float): Multiplier of (pot + to_call), such as 0.5 for a half-pot bet.
        Return Value:
            Tuple[str, int]: ('raise', sized_raise_by) or ('call', 0) if unable to raise.
        Examples:
            >>> s = State(pot=100, small_blind=5, min_raise=10)
            >>> pot_raise(s, {"can_raise": True, "to_call": 0, "room": 200}, 0.5)
            ('raise', 50)
        """
        size = int((state.pot + ctx["to_call"]) * fraction)
        size = (size // state.small_blind) * state.small_blind
        return make_raise(state, ctx, size)


    def raise_to(state: State, ctx: dict, target_total: float) -> Tuple[str, int]:
        """
```

```
        Function Purpose:
            Convert a target total bet level into a legal raise increment above the current highest bet.
        Parameters:
            state (State): The current game state.
            ctx (dict): Bot context dictionary containing 'highest'.
            target_total (float): Desired total bet amount for the current round.
        Return Value:
            Tuple[str, int]: ('raise', raise_increment) or ('call', 0) if unable to raise.
        Examples:
            >>> s = State(min_raise=10)
            >>> raise_to(s, {"can_raise": True, "highest": 10, "room": 100}, 30)
            ('raise', 20)
        """
        return make_raise(state, ctx, int(target_total) - ctx["highest"])


    def shove(ctx: dict) -> Tuple[str, int]:
        """
        Function Purpose:
            Move all-in by raising all remaining chips if raising is legal, otherwise calling.
        Parameters:
            ctx (dict): Bot context dictionary containing 'can_raise' and 'room'.
        Return Value:
            Tuple[str, int]: ('raise', room) if raising is legal, else ('call', 0).
        Examples:
            >>> shove({"can_raise": True, "room": 85})
            ('raise', 85)
        """
        return ("raise", ctx["room"]) if ctx["can_raise"] else ("call", 0)


    def preflop_action(state: State, idx: int, profile: dict, ctx: dict) -> Tuple[str, int]:
        """
        Function Purpose:
            Select a bot's pre-flop action ('fold', 'call', or 'raise') using Chen score, position, and stack size.
        Parameters:
            state (State): The current game state.
            idx (int): Seat index of the acting bot.
            profile (dict): Difficulty tuning dictionary from BOT_PROFILES.
            ctx (dict): Bot context dictionary from bot_context().
        Return Value:
            Tuple[str, int]: Chosen (action_name, raise_amount) pair.
        Examples:
            >>> s = State(players=[Player("U", 1000), Player("B1", 1000, hole_cards=["A♠", "A♥"])], big_blind=10, small_blin
            >>> ctx = bot_context(s, 1, BOT_PROFILES["Hard"])
            >>> preflop_action(s, 1, BOT_PROFILES["Hard"], ctx)[0]
            'raise'
        """
        bot = state.players[idx]
        bb = state.big_blind
        to_call, highest = ctx["to_call"], ctx["highest"]
        n_active = len(ctx["opps"]) + 1
        late = ctx["lateness"]

        chen = chen_score(bot.hole_cards)
        # ADVANCED TECHNIQUE (random.gauss): adds Gaussian (normal distribution) noise
        # so Easy/Medium bots occasionally misjudge marginal starting hands.
        if profile["pre_noise"] > 0:
            chen += random.gauss(0, profile["pre_noise"])
        open_thr = 7.5 - 3.0 * late + 0.3 * (n_active - 3) + profile["pre_tight"]
        stack_bb = (bot.chips + bot.current_bet) / bb
        raised_pot = highest > bb
        check_or_fold = ("call", 0) if to_call == 0 else ("fold", 0)

        if stack_bb <= 10:
            push_thr = 9.5 if raised_pot else 6.0
            return shove(ctx) if chen >= push_thr else check_or_fold

        if not raised_pot:
            limpers = max(0, sum(1 for j in ctx["opps"] if state.players[j].current_bet >= bb) - 1)
            if chen >= open_thr + profile["raise_extra"]:
                return raise_to(state, ctx, bb * (3 + limpers))
            if to_call == 0 or (chen >= open_thr - profile["limp_gap"] and to_call <= bb):
                return ("call", 0)
```

```
            return ("fold", 0)

        is_bb = idx == (state.dealer + 2) % len(state.players)
        price = to_call / (state.pot + to_call)
        premium = 13.5 if state.raises_this_street >= 2 else profile["threebet"] + ctx["agg"]
        if chen >= premium:
            return raise_to(state, ctx, highest * (3 if late >= 0.5 else 3.5))
        if (state.raises_this_street == 1 and late >= 0.75 and 6 <= chen < premium
                and random.random() < profile["bluff3"]):
            return raise_to(state, ctx, highest * 3)

        call_thr = open_thr + 0.8 + 0.6 * max(0.0, to_call / bb - 2) + ctx["agg"] - profile["call_loose"]
        if price < 0.2:
            call_thr -= 1.5
        if is_bb and to_call <= 3 * bb:
            call_thr -= 1.5
        if to_call >= 0.4 * bot.chips:
            call_thr = max(call_thr, 11.0)
        call_thr = min(call_thr, 14.0)
        return ("call", 0) if chen >= call_thr else ("fold", 0)


    def postflop_action(state: State, idx: int, profile: dict, ctx: dict) -> Tuple[str, int]:
        """
        Function Purpose:
            Select a bot's post-flop action ('fold', 'call', or 'raise') using Monte Carlo win equity and pot odds.
        Parameters:
            state (State): The current game state (stage in 'flop', 'turn', or 'river').
            idx (int): Seat index of the acting bot.
            profile (dict): Difficulty tuning dictionary from BOT_PROFILES.
            ctx (dict): Bot context dictionary from bot_context().
        Return Value:
            Tuple[str, int]: Chosen (action_name, raise_amount) pair.
        Examples:
            >>> s = State(players=[Player("U", 1000), Player("B1", 1000, hole_cards=["A♠", "A♥"])], community_cards=["A♦", "
            >>> ctx = bot_context(s, 1, BOT_PROFILES["Hard"])
            >>> postflop_action(s, 1, BOT_PROFILES["Hard"], ctx)[0]
            'raise'
        """
        bot = state.players[idx]
        to_call, room, late = ctx["to_call"], ctx["room"], ctx["lateness"]
        opps = ctx["opps"]
        n_opp = len(opps)

        floors = []
        for j in opps:
            floor = 0.0
            if profile["read_floor"] > 0 and state.raised_flags[j]:
                rate = raise_rate(state, j)
                floor = profile["read_floor"] + (1.5 if rate < 0.15 else (-1.5 if rate > 0.40 else 0.0))
            floors.append(floor)
        equity = estimate_equity(bot.hole_cards, state.community_cards, floors, profile["sims"])
        if profile["noise"] > 0:
            equity += random.gauss(0, profile["noise"])

        fair = 1.0 / (n_opp + 1)
        raise_at = fair + profile["raise_k"] * (1.0 - fair)
        pot_odds = to_call / (state.pot + to_call) if to_call > 0 else 0.0
        need = pot_odds * profile["fold_margin"] * ctx["agg_scale"]
        if profile["reads"] and to_call > 0 and state.stage in ("flop", "turn") and room > 2 * state.pot:
            need -= 0.03
        if to_call > 0 and to_call >= 0.6 * bot.chips:
            need = min(need, pot_odds)
        check_or_fold = ("call", 0) if to_call == 0 else ("fold", 0)

        if (bot.chips + bot.current_bet) / state.big_blind <= 8:
            if equity >= raise_at - 0.08:
                return shove(ctx)
            return ("call", 0) if (to_call == 0 or equity >= need) else check_or_fold

        if to_call == 0:
            if equity >= raise_at:
                if equity > 0.88 and state.stage != "river" and random.random() < profile["trap"]:
                    return ("call", 0)
```

```
            fraction = 0.5 if equity < 0.65 else (0.7 if equity < 0.8 else 1.0)
            return pot_raise(state, ctx, fraction)
        if (state.stage == "flop" and state.preflop_aggressor == idx and n_opp <= 2
                and random.random() < profile["cbet"]):
            return pot_raise(state, ctx, 0.5)
        if n_opp <= 2 and 0.2 <= equity < raise_at and random.random() < profile["bluff"] * (0.5 + late):
            return pot_raise(state, ctx, 0.55)
        return ("call", 0)

    if equity < need:
        if (state.stage != "river" and n_opp == 1
                and random.random() < profile["bluff"] * 0.4):
            return pot_raise(state, ctx, 0.8)
        return ("fold", 0)
    strong = equity >= raise_at + 0.05
    if state.raises_this_street >= 2:
        strong = equity >= 0.85
    if strong:
        return pot_raise(state, ctx, 0.75 if equity < 0.8 else 1.0)
    if (state.stage in ("flop", "turn") and n_opp == 1 and 0.3 <= equity < raise_at
            and random.random() < profile["semi"]):
        return pot_raise(state, ctx, 0.75)
    return ("call", 0)


def choose_bot_action(state: State, idx: int) -> Tuple[str, int]:
    """
    Function Purpose:
        Choose the action and raise size for AI bot idx according to difficulty and current betting street.
    Parameters:
        state (State): The current game state.
        idx (int): Seat index of the acting bot.
    Return Value:
        Tuple[str, int]: ('fold' | 'call' | 'raise', raise_by) for the bot.
    Examples:
        >>> s = State(players=[Player("U", 1000), Player("B1", 1000, hole_cards=["7♦", "2♣"], current_bet=0)], stage="pr
        >>> s.players[0].current_bet = 30
        >>> choose_bot_action(s, 1)
        ('fold', 0)
    """
    profile = BOT_PROFILES.get(state.difficulty, BOT_PROFILES["Medium"])
    ctx = bot_context(state, idx, profile)
    if state.stage == "preflop":
        return preflop_action(state, idx, profile, ctx)
    return postflop_action(state, idx, profile, ctx)


# ------------------------------------------------------------------------
# Pot awarding and showdown resolution
# ------------------------------------------------------------------------
def build_side_pots(state: State) -> List[Tuple[int, List[int]]]:
    """
    Function Purpose:
        Split total hand contributions into main and side pots when players are all-in for different amounts.
    Parameters:
        state (State): The current game state containing contributed amounts and players.
    Return Value:
        List[Tuple[int, List[int]]]: List of (pot_chip_amount, eligible_player_indices) tuples.
    Examples:
        >>> s = State(players=[Player("A", 0), Player("B", 50)], contributed=[50, 100])
        >>> build_side_pots(s)
        [(100, [0, 1]), (50, [1])]
    """
    pots = []
    previous = 0
    # Set comprehension extracts unique positive contribution thresholds in ascending order
    for level in sorted({c for c in state.contributed if c > 0}):
        contributors = [i for i, c in enumerate(state.contributed) if c >= level]
        amount = (level - previous) * len(contributors)
        eligible = [i for i in contributors if not state.players[i].is_folded]
        if not eligible:
            eligible = [i for i, p in enumerate(state.players) if not p.is_folded]
        pots.append((amount, eligible))
        previous = level
```

```
    return pots


def finish_hand(state: State, winnings: List[int], reveal: bool, headline: str,
                hand_names: List[str], winning_cards: List[str], winning_name: str) -> Page:
    """
    Function Purpose:
        Pay out chip winnings, reset the pot, update session stats, and return the showdown Page.
    Parameters:
        state (State): The current game state.
        winnings (List[int]): Chips won by each player seat index.
        reveal (bool): Whether hole cards should be revealed on the showdown page.
        headline (str): Summary banner text describing the winner(s).
        hand_names (List[str]): Evaluated hand name per player (empty string if folded/unshown).
        winning_cards (List[str]): Best 5 cards of the winning hand.
        winning_name (str): Name of the winning 5-card hand.
    Return Value:
        Page: Rendered showdown_page(state) displaying the hand outcome.
    Examples:
        >>> s = State(players=[Player("User", 950), Player("Bot 1", 950)], pot=100, contributed=[50, 50])
        >>> page = finish_hand(s, [100, 0], False, "User wins", ["", ""], [], "")
        >>> s.players[0].chips
        1050
    """
    total = state.pot
    for i, amount in enumerate(winnings):
        state.players[i].chips += amount
    state.pot = 0
    state.winnings = winnings
    state.reveal = reveal
    state.result_headline = headline
    state.hand_names = hand_names
    state.winning_cards = winning_cards
    state.winning_hand_name = winning_name
    state.last_pot = total
    state.stage = "showdown"
    state.hands_played += 1
    if winnings[0] > 0:
        state.hands_won += 1
        state.biggest_pot = max(state.biggest_pot, total)
    state.peak_chips = max(state.peak_chips, state.players[0].chips)
    return showdown_page(state)


def award_uncontested(state: State, winner_idx: int) -> Page:
    """
    Function Purpose:
        Award the entire pot to the sole remaining player when all other players have folded.
    Parameters:
        state (State): The current game state.
        winner_idx (int): Seat index of the single un-folded player.
    Return Value:
        Page: Rendered showdown_page(state) showing the uncontested win.
    Examples:
        >>> s = State(players=[Player("User", 980), Player("Bot 1", 980, is_folded=True)], pot=40, contributed=[20, 20])
        >>> page = award_uncontested(s, 0)
        >>> s.players[0].chips
        1020
    """
    n = len(state.players)
    winnings = [0] * n
    winnings[winner_idx] = state.pot
    winner = state.players[winner_idx]
    headline = "★ " + winner.name + " wins " + str(state.pot) + " chips - everyone else folded ★"
    return finish_hand(state, winnings, False, headline, [""] * n, [], "")


def resolve_showdown(state: State) -> Page:
    """
    Function Purpose:
        Evaluate all remaining players' 7-card hands at showdown, distribute main and side pots,
        and display the showdown results page.
    Parameters:
        state (State): The current game state with 5 community cards dealt.
```

```
        Return Value:
            Page: Rendered showdown_page(state) with revealed cards and payouts.
        Examples:
            >>> p0 = Player("User", 900, hole_cards=["A♠", "A♥"])
            >>> p1 = Player("Bot 1", 900, hole_cards=["K♠", "K♥"])
            >>> s = State(players=[p0, p1], community_cards=["2♣", "5♦", "8♠", "J♥", "3♣"], pot=200, contributed=[100, 100])
            >>> page = resolve_showdown(s)
            >>> s.players[0].chips
            1100
        """
        n = len(state.players)
        scored = {}
        for i, p in enumerate(state.players):
            if not p.is_folded:
                scored[i] = best_hand(p.hole_cards + state.community_cards)

        winnings = [0] * n
        for amount, eligible in build_side_pots(state):
            in_running = [i for i in eligible if i in scored]
            top = max(scored[i][0] for i in in_running)
            winners = [i for i in in_running if scored[i][0] == top]
            # Built-in divmod() computes integer split share and any indivisible odd-chip remainder
            share, extra = divmod(amount, len(winners))
            for w in winners:
                winnings[w] += share
            winnings[winners[0]] += extra

        hand_names = [scored[i][2] if i in scored else "" for i in range(n)]
        best_idx = max(scored, key=lambda i: scored[i][0])
        best_score_value = scored[best_idx][0]
        top_players = [i for i in scored if scored[i][0] == best_score_value]
        paid = [i for i in range(n) if winnings[i] > 0]

        best_name = scored[best_idx][2]
        if len(paid) == 1:
            headline = "★ " + state.players[paid[0]].name + " wins " + str(state.pot) + " chips with " + best_name + " ★"
        elif set(paid) == set(top_players):
            names = " and ".join(state.players[i].name for i in paid)
            headline = "★ Split pot: " + names + " tie with " + best_name + " ★"
        else:
            parts = [state.players[i].name + " wins " + str(winnings[i]) for i in paid]
            headline = "★ " + ", ".join(parts) + " (best hand: " + best_name + ") ★"
        return finish_hand(state, winnings, True, headline, hand_names, scored[best_idx][1], scored[best_idx][2])


    def check_progress(state: State) -> Optional[Page]:
        """
        Function Purpose:
            Evaluate whether the hand has ended due to folds or if the current betting round is complete.
        Parameters:
            state (State): The current game state.
        Return Value:
            Optional[Page]: Next Page if the hand moved to a new street or showdown, or None if betting continues.
        Examples:
            >>> s = State(players=[Player("User", 100), Player("Bot 1", 100, is_folded=True)], pot=20, contributed=[10, 10])
            >>> check_progress(s) is not None
            True
        """
        active = [i for i, p in enumerate(state.players) if not p.is_folded]
        if len(active) == 1:
            return award_uncontested(state, active[0])

        if betting_round_complete(state):
            if len(state.community_cards) < 5:
                return advance_street(state)
            return resolve_showdown(state)
        return None


    # ---------------------------------------------------------------------------
    # Routes
    # ---------------------------------------------------------------------------
    @route
    def index(state: State) -> Page:
```

```
    """
    Function Purpose:
        Entry route that renders the appropriate Page for the current game stage.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: welcome_page, showdown_page, game_over_page, or table_page depending on state.stage.
    Examples:
        >>> s = State(stage="setup")
        >>> type(index(s)).__name__
        'Page'
    """
    if state.stage == "setup":
        return welcome_page(state)
    if state.stage == "showdown":
        return showdown_page(state)
    if state.stage == "gameover":
        return game_over_page(state)
    return table_page(state)


@route
def start_new_game(state: State, player_name: str, starting_stack: str, difficulty: str) -> Page:
    """
    Function Purpose:
        Initialize a new 5-player poker session with custom name, starting chips, and AI difficulty.
    Parameters:
        state (State): The current game state to reset.
        player_name (str): Display name entered by the human player.
        starting_stack (str): Selected starting chip count string (e.g., '1000').
        difficulty (str): Selected AI difficulty ('Easy', 'Medium', or 'Hard').
    Return Value:
        Page: The active table_page (or showdown_page if all bots fold pre-flop) after dealing Hand #1.
    Examples:
        >>> s = State()
        >>> page = start_new_game(s, "Alice", "1000", "Medium")
        >>> s.players[0].name
        'Alice'
    """
    try:
        stack = int(starting_stack)
    except ValueError:
        stack = 1000
    name = player_name.strip() or "User"

    state.starting_stack = stack
    state.difficulty = difficulty if difficulty in BOT_PROFILES else "Medium"
    state.big_blind = max(2, (stack // 50) // 2 * 2)
    state.small_blind = state.big_blind // 2
    state.min_raise = state.big_blind

    state.players = [Player(name, stack)] + [Player("Bot " + str(k), stack) for k in range(1, NUM_PLAYERS)]
    n = len(state.players)
    state.community_cards = []
    state.pot = 0
    state.current_turn_index = 0
    state.deck = new_deck()
    state.dealer = n - 1
    state.hand_number = 0
    state.acted = [False] * n
    state.contributed = [0] * n
    state.raised_flags = [False] * n
    state.preflop_aggressor = -1
    state.opp_raises = [0] * n
    state.opp_actions = [0] * n
    state.action_log = []
    state.message = ""
    state.hands_played = 0
    state.hands_won = 0
    state.biggest_pot = 0
    state.rebuys = 0
    state.peak_chips = stack
    return deal_hand(state)
```

```
@route
def deal_hand(state: State) -> Page:
    """
    Function Purpose:
        Start a new hand by rotating the dealer button, dealing 2 hole cards per player,
        posting blinds, and running initial bot actions.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: game_over_page if the human has 0 chips, otherwise the table_page or showdown_page.
    Examples:
        >>> s = State(players=[Player("User", 0), Player("Bot 1", 1000)])
        >>> page = deal_hand(s)
        >>> s.stage
        'gameover'
    """
    human = state.players[0]
    if human.chips <= 0:
        state.stage = "gameover"
        return game_over_page(state)

    n = len(state.players)
    for bot in state.players[1:]:
        if bot.chips <= 0:
            bot.chips = state.starting_stack

    state.hand_number += 1
    state.action_log = []
    state.message = ""
    state.deck = new_deck()
    state.community_cards = []
    state.pot = 0
    state.acted = [False] * n
    state.contributed = [0] * n
    state.min_raise = state.big_blind
    state.raises_this_street = 0
    state.raised_flags = [False] * n
    state.preflop_aggressor = -1
    state.dealer = (state.dealer + 1) % n

    for p in state.players:
        p.hole_cards = []
        p.current_bet = 0
        p.is_folded = False
    for _ in range(2):
        for p in state.players:
            p.hole_cards.append(state.deck.pop())

    small_idx = (state.dealer + 1) % n
    big_idx = (state.dealer + 2) % n
    post_blind(state, small_idx, state.small_blind, "small blind")
    post_blind(state, big_idx, state.big_blind, "big blind")

    state.stage = "preflop"
    state.current_turn_index = big_idx
    state.current_turn_index = next_actor(state)
    return trigger_bot_turns(state)


@route
def player_action(state: State, action_choice: str, raise_amount: str, auto_muck: bool) -> Page:
    """
    Function Purpose:
        Validate and execute the human player's chosen action (Fold, Check/Call, or Raise).
    Parameters:
        state (State): The current game state.
        action_choice (str): Selected dropdown action ('Check/Call', 'Raise', or 'Fold').
        raise_amount (str): Text box input specifying chips to add on top of the call.
        auto_muck (bool): Checkbox value indicating whether to hide losing human hands at showdown.
    Return Value:
        Page: Updated table_page (with validation warning if invalid) or showdown_page.
    Examples:
        >>> s = State()
```

```
        >>> _ = start_new_game(s, "User", "1000", "Medium")
        >>> page = player_action(s, "Fold", "10", False)
        >>> type(page).__name__
        'Page'
    """
    state.auto_muck = auto_muck
    human = state.players[0]
    if state.stage not in BETTING_STAGES or state.current_turn_index != 0 or not can_act(human):
        return index(state)

    state.message = ""
    to_call = max(0, highest_bet(state) - human.current_bet)

    if action_choice == "Fold":
        apply_action(state, 0, "fold")
    elif action_choice == "Raise":
        try:
            raise_by = int(raise_amount.strip())
        except ValueError:
            state.message = "Raise amount must be a whole number of chips, for example " + str(state.min_raise) + "."
            return table_page(state)

        if raise_by >= state.min_raise and to_call + raise_by <= human.chips:
            apply_action(state, 0, "raise", raise_by)
        else:
            max_raise = human.chips - to_call
            if max_raise < state.min_raise:
                state.message = ("You don't have enough chips to raise (minimum raise is "
                                 + str(state.min_raise) + "). Choose Check/Call or Fold.")
            else:
                state.message = ("Invalid raise of " + str(raise_by) + ": enter between "
                                 + str(state.min_raise) + " and " + str(max_raise) + " chips.")
            return table_page(state)
    else:
        apply_action(state, 0, "call")

    state.current_turn_index = next_actor(state)
    return trigger_bot_turns(state)


@route
def trigger_bot_turns(state: State) -> Page:
    """
    Function Purpose:
        Advance AI opponent turns in sequence until it is the human's turn, a street ends, or the hand concludes.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: table_page when awaiting human input, or showdown_page when the hand finishes.
    Examples:
        >>> s = State(players=[Player("User", 1000)], stage="preflop", current_turn_index=0, acted=[False])
        >>> type(trigger_bot_turns(s)).__name__
        'Page'
    """
    for _ in range(300):
        page = check_progress(state)
        if page is not None:
            return page
        idx = state.current_turn_index
        if not can_act(state.players[idx]):
            state.current_turn_index = next_actor(state)
            continue
        if idx == 0:
            return table_page(state)
        action, raise_by = choose_bot_action(state, idx)
        apply_action(state, idx, action, raise_by)
        state.current_turn_index = next_actor(state)
    return table_page(state)


@route
def advance_street(state: State) -> Page:
    """
    Function Purpose:
```

```
        Reset street bets, burn one card, deal Flop/Turn/River community cards, and resume betting.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: Next table_page for the new betting street, or showdown_page after the river.
    Examples:
        >>> s = State(players=[Player("U", 1000), Player("B1", 1000)], deck=new_deck(), dealer=0, big_blind=10)
        >>> page = advance_street(s)
        >>> len(s.community_cards)
        3
    """
    cards_out = len(state.community_cards)
    if cards_out >= 5:
        return resolve_showdown(state)

    for p in state.players:
        p.current_bet = 0
    state.acted = [False] * len(state.players)
    state.min_raise = state.big_blind
    state.raises_this_street = 0

    state.deck.pop()
    if cards_out == 0:
        state.stage = "flop"
        for _ in range(3):
            state.community_cards.append(state.deck.pop())
    elif cards_out == 3:
        state.stage = "turn"
        state.community_cards.append(state.deck.pop())
    else:
        state.stage = "river"
        state.community_cards.append(state.deck.pop())

    state.current_turn_index = state.dealer
    state.current_turn_index = next_actor(state)
    return trigger_bot_turns(state)


@route
def reset_stack(state: State) -> Page:
    """
    Function Purpose:
        Re-buy a fresh starting_stack for the human player after running out of chips and deal a new hand.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: Rendered table_page (or showdown_page) for the newly dealt hand.
    Examples:
        >>> s = State(players=[Player("User", 0), Player("Bot 1", 1000)], starting_stack=1000)
        >>> page = reset_stack(s)
        >>> s.rebuys
        1
    """
    human = state.players[0]
    if human.chips <= 0:
        human.chips = state.starting_stack
        state.rebuys += 1
        state.peak_chips = max(state.peak_chips, human.chips)
        state.message = ""
        return deal_hand(state)
    return index(state)


@route
def show_rules(state: State) -> Page:
    """
    Function Purpose:
        Navigate to the Rules & Hand Rankings reference page without altering game progress.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: Rendered rules_page(state).
    Examples:
        >>> type(show_rules(State())).__name__
```

```
        'Page'
    """
    return rules_page(state)


@route
def return_to_start(state: State) -> Page:
    """
    Function Purpose:
        Exit the current table and return to the initial welcome/setup screen.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: Rendered welcome_page(state) with state.stage set to 'setup'.
    Examples:
        >>> s = State(stage="preflop")
        >>> page = return_to_start(s)
        >>> s.stage
        'setup'
    """
    state.stage = "setup"
    state.message = ""
    return welcome_page(state)


# ------------------------------------------------------------------------
# Pages
# ------------------------------------------------------------------------
def welcome_page(state: State) -> Page:
    """
    Function Purpose:
        Construct the initial game configuration Page with inputs for player name, stack size, and difficulty.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: Drafter Page object for the welcome/setup screen.
    Examples:
        >>> type(welcome_page(State())).__name__
        'Page'
    """
    return Page(state, [
        Header("Texas Hold'em Poker", 1),
        "Play against four computer opponents. Set up your game below.",
        HorizontalRule(),
        "Your name:",
        TextBox("player_name", "User"),
        "Starting chip stack:",
        SelectBox("starting_stack", ["500", "1000", "2000", "5000"], "1000"),
        "Opponent difficulty:",
        SelectBox("difficulty", ["Easy", "Medium", "Hard"], "Medium"),
        "Blinds are set to 1/50 (big) and 1/100 (small) of your starting stack.",
        LineBreak(),
        Button("Start Game", start_new_game),
        Button("Rules & Hand Rankings", show_rules),
    ])


def table_page(state: State) -> Page:
    """
    Function Purpose:
        Construct the main active poker table Page with visual table layout, action history, and betting inputs.
    Parameters:
        state (State): The current game state.
    Return Value:
        Page: Drafter Page object representing the live poker table.
    Examples:
        >>> s = State(players=[Player("P" + str(k), 1000) for k in range(5)], stage="preflop")
        >>> type(table_page(s)).__name__
        'Page'
    """
    human = state.players[0]
    to_call = max(0, highest_bet(state) - human.current_bet)
    max_raise = human.chips - to_call
```

```
    content = [Header("Hand #" + str(state.hand_number) + " - " + STAGE_LABELS.get(state.stage, state.stage), 2)]
    if state.message:
        content.append("⚠ " + state.message)
    content.append("Opponents: " + state.difficulty + "   |   Dealer, small blind and big blind rotate every hand")
    content.append(table_layout(state))

    if len(human.hole_cards) + len(state.community_cards) >= 5 and not human.is_folded:
        content.append("Your best hand right now: " + best_hand(human.hole_cards + state.community_cards)[2])

    if state.action_log:
        content.append(HorizontalRule())
        content.append("Recent action:")
        for line in state.action_log[-6:]:
            content.append("  • " + line)
    content.append(HorizontalRule())

    if state.current_turn_index == 0 and can_act(human):
        if to_call == 0:
            content.append("Your turn - nothing to call, you may check.")
        else:
            content.append("Your turn - " + str(min(to_call, human.chips)) + " to call.")
        if max_raise >= state.min_raise:
            content.append("Raise amount = chips added on top of the call. Minimum " + str(state.min_raise)
                           + ", maximum " + str(max_raise) + ".")
        else:
            content.append("You can't raise right now (not enough chips) - check/call or fold.")
        content.append("Action:")
        content.append(SelectBox("action_choice", ["Check/Call", "Raise", "Fold"], "Check/Call"))
        content.append("Raise amount (only used when Raise is selected):")
        content.append(TextBox("raise_amount", str(state.min_raise)))
        content.append("Auto-muck my losing hands at showdown:")
        content.append(CheckBox("auto_muck", state.auto_muck))
        content.append(LineBreak())
        content.append(Button("Submit Action", player_action))
    else:
        content.append(Button("Continue", trigger_bot_turns))
    content.append(Button("Rules & Hand Rankings", show_rules))
    content.append(Button("Quit to Start Screen", return_to_start))
    return Page(state, content)


def showdown_page(state: State) -> Page:
    """
    Function Purpose:
        Construct the post-hand Showdown / Hand Results Page showing payouts, revealed hands, and next-hand button.
    Parameters:
        state (State): The current game state at the end of a hand.
    Return Value:
        Page: Drafter Page object summarizing the completed hand's results.
    Examples:
        >>> s = State(players=[Player("P" + str(k), 1000) for k in range(5)], stage="showdown", winnings=[0]*5, contribu
        >>> type(showdown_page(s)).__name__
        'Page'
    """
    content = [Header("Hand Results - Hand #" + str(state.hand_number), 1),
               Header(state.result_headline, 2),
               "Pot awarded: " + str(state.last_pot) + " chips",
               table_layout(state, reveal=state.reveal)]

    rows = [["Player", "Hole cards", "Hand", "Result", "Chips now"]]
    for i, p in enumerate(state.players):
        won = state.winnings[i] if i < len(state.winnings) else 0
        if p.is_folded:
            cards, hand = "Folded", "-"
        elif not state.reveal:
            cards = cards_text(p.hole_cards) if i == 0 else "Not shown"
            hand = "-"
        elif i == 0 and won == 0 and state.auto_muck:
            cards, hand = "Mucked (hidden)", "-"
        else:
            cards, hand = cards_text(p.hole_cards), state.hand_names[i]
        if won > 0:
            result = "★ WINNER  +" + str(won)
        elif state.contributed[i] > 0:
```

```
            result = "Lost " + str(state.contributed[i])
        else:
            result = "-"
        rows.append([p.name, cards, hand, result, str(p.chips)])
    content.append(Table(rows))

    if state.reveal and state.winning_cards:
        content.append("Winning 5-card hand: " + state.winning_hand_name)
        content.append(box(card_row(state.winning_cards, 62), bg=FELT, pad="8px", align="center"))
    elif not state.reveal:
        content.append("No showdown - the last player standing wins the pot.")

    content.append(HorizontalRule())
    if state.players[0].chips <= 0:
        content.append("You're out of chips.")
        content.append(Button("Continue", deal_hand))
    else:
        content.append(Button("Deal Next Hand", deal_hand))
    content.append(Button("Rules & Hand Rankings", show_rules))
    content.append(Button("Quit to Start Screen", return_to_start))
    return Page(state, content)


def game_over_page(state: State) -> Page:
    """
    Function Purpose:
        Construct the Game Over Page showing session statistics along with rebuy and new-game controls.
    Parameters:
        state (State): The current game state after the human player reaches 0 chips.
    Return Value:
        Page: Drafter Page object displaying session stats and restart options.
    Examples:
        >>> s = State(players=[Player("User", 0)], stage="gameover")
        >>> type(game_over_page(s)).__name__
        'Page'
    """
    played = state.hands_played
    rate = (str(round(100 * state.hands_won / played)) + "%") if played else "n/a"
    invested = state.starting_stack * (1 + state.rebuys)
    stats = [
        ["Session statistic", "Value"],
        ["Hands played", str(played)],
        ["Hands won", str(state.hands_won)],
        ["Win rate", rate],
        ["Biggest pot won", str(state.biggest_pot)],
        ["Peak chip count", str(state.peak_chips)],
        ["Re-buys used", str(state.rebuys)],
        ["Net result", str(state.players[0].chips - invested) + " chips"],
    ]
    return Page(state, [
        Header("Game Over", 1),
        "You're out of chips. You can re-buy and keep playing, or start over.",
        Table(stats),
        HorizontalRule(),
        Button("Re-buy " + str(state.starting_stack) + " chips and keep playing", reset_stack),
        HorizontalRule(),
        Header("Or start a brand-new game", 3),
        "Your name:",
        TextBox("player_name", state.players[0].name),
        "Starting chip stack:",
        SelectBox("starting_stack", ["500", "1000", "2000", "5000"], str(state.starting_stack)),
        "Opponent difficulty:",
        SelectBox("difficulty", ["Easy", "Medium", "Hard"], state.difficulty),
        Button("Start Over", start_new_game),
        Button("Rules & Hand Rankings", show_rules),
        Button("Back to Start Screen", return_to_start),
    ])


def rules_page(state: State) -> Page:
    """
    Function Purpose:
        Construct the Rules & Hand Rankings guide Page explaining gameplay flow, betting, and hand tiers.
    Parameters:
```

```
        state (State): The current game state.
    Return Value:
        Page: Drafter Page object containing the Texas Hold'em rules and hand ranking table.
    Examples:
        >>> type(rules_page(State())).__name__
        'Page'
    """
    return Page(state, [
        Header("Rules & Hand Rankings", 1),
        Header("How a hand works", 3),
        "1. Each player is dealt two private hole cards. The small blind and big blind post forced bets.",
        "2. Pre-flop betting: starting left of the big blind, each player may check/call, raise, or fold.",
        "3. The flop (3 community cards) is dealt, followed by another betting round.",
        "4. The turn (1 card) is dealt, followed by betting.",
        "5. The river (1 card) is dealt, followed by the final betting round.",
        "6. Showdown: remaining players reveal their cards. The best five-card hand, made from any "
        "combination of your two hole cards and the five community cards, wins the pot.",
        "If everyone else folds, the last player wins the pot immediately without showing cards.",
        Header("Betting in this game", 3),
        "Check/Call: match the current bet (or check if there is none). "
        "Raise: type how many chips to add on top of the call - at least the minimum raise shown. "
        "Fold: give up your hand. If a player is all-in for less than the others bet, "
        "a side pot is created so they can only win what they matched.",
        "Auto-muck: tick the box to hide your hand at showdown when you lose.",
        Header("Hand rankings (best to worst)", 3),
        Table([["Rank", "Hand", "Description", "Example"]]
              + [[r[0], r[1], r[2], colorize(r[3])] for r in HAND_RANKINGS]),
        "When two players have the same hand type, the higher cards win; if all five cards play "
        "equally, the pot is split.",
        LineBreak(),
        Button("Back to the Game", index),
        Button("Back to Start Screen", return_to_start),
    ])


start_server(State())
```
